

Лабораторная работа 1
**СОЗДАНИЕ БАЗ ДАННЫХ В СУБД MYSQL ПРИ ПОМОЩИ
ИНСТРУМЕНТАРИЯ DBFORGE STUDIO FOR MYSQL**

Цель работы: ознакомиться с процессом создания базы данных в СУБД **MySQL** с использованием графического клиента **dbForge Studio for MySQL**. изучить особенности создания и настройки таблиц; ознакомиться с механизмами триггеров и представлений; реализовать базу данных для ведения складского учета; заполнить таблицы базы данных.

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Ознакомиться постановкой задачи на проектирование базы данных.
2. Ознакомиться с теоретическими сведениями.
3. Запустить сервер *MySQL*.
4. Настроить подключение к серверу.
5. Изучить, что хранится в служебных базах данных.
6. Создать базу данных.
7. Ознакомиться с используемыми кодировками и подсистемами хранения таблиц.
8. Выполнить создание таблиц базы.
9. Ознакомиться с используемыми в *MySQL* типами данных.
10. Выполнить настройку индексов.
11. Организовать связь между таблицами базы данных.
12. Заполнить таблицу **Типы товаров**.
13. Добавить одну запись в таблицу **Товары**.
14. Реализовать триггеры для управления списком товаров на складе.
15. Добавить новые записи в таблицу **Товары**.
16. Заполнить остальные таблицы базы данных.
17. Создать три представления *view_storage*, *view_goods_catalog* и *view_goods_operations*.
18. Выполнить создание резервной копии базы данных.
19. Ознакомиться с содержимым резервной копии БД.
20. Ответить на контрольные вопросы.

ЗАДАЧА ПРОЕКТИРОВАНИЯ БАЗЫ ДАННЫХ

В рамках лабораторного практикума будет осуществляться разработка приложения, предназначенное для складского учета товаров на примере магазина компьютерной техники. База данных будет состоять из шести таблиц: **Товары**, **Типы товаров**, **Склад товаров**, **Поставщики**, **Операции с товарами**, и **Состав операций** (табл. 1.1).

Таблица 1.1

Структура таблиц базы данных

Название поля	Тип поля	Дополнительные параметры	Ключ / индекс	Описание
<i>goods_catalog (Товары)</i>				
IDgc	INT(11)	Автоинкремент	Ключ	Код товара
gcName	VARCHAR(100)	Не пустой, Уникальный	Индекс	Наименование товара
IDtg	INT(11)	Не пустой	ВК, Индекс	Код типа товара
gcDescription	VARCHAR(255)	По умолчанию: NULL	–	Описание товара
gcPhoto	LONGBLOB	По умолчанию: NULL	–	Фотография товара
gcCost	DECIMAL(10,2)	По умолчанию: 1	–	Стоимость единицы товара
<i>type_goods (Типы товаров)</i>				
IDtg	INT(11)	Автоинкремент	Ключ	Код типа товара
tgName	VARCHAR(50)	Не пустой	–	Наименование типа товара
<i>suppliers_catalog (Поставщики)</i>				
IDsc	INT	Автоинкремент	Ключ	Код поставщика
scName	VARCHAR(50)	Не пустой, Уникальный	Индекс	Название поставщика
scAddress	VARCHAR(100)	По умолчанию: NULL	–	Адрес поставщика
scPhone	VARCHAR(15)	По умолчанию: NULL	–	Телефон поставщика
scEmail	VARCHAR(50)	Уникальный По умолчанию: NULL	Индекс	e-mail поставщика
<i>operations (Операции с товарами)</i>				
IDoperation	INT(11)	Автоинкремент	Ключ	Код операции
oDateTime	DATETIME	Не пустой	Индекс	Дата операции
oIsSale	TINYINT(1)	По умолчанию: 1	–	Тип операции (1-продажа 0-поставка)
IDsc	INT(11)	По умолчанию: NULL	ВК, Индекс	Код поставщика
<i>op_goods_list (Состав операций)</i>				
IDoperation	INT(11)	Не пустой	Ключ ВК,	Код операции
IDgc	INT(11)	Не пустой	Ключ ВК, Индекс	Код товара
ocCount	INT(11)	По умолчанию: 1	–	Количество товара

Название Поля	Тип поля	Дополнительные параметры	Ключ / индекс	Описание
storage (Склад товаров)				
IDstorage	INT(11)	Автоинкремент	Ключ	Код складской записи
IDgc	INT(11)	Не пустой	ВК, Индекс	Код товара
sCount	INT(11)	Не пустой По умолчанию 0	—	Количество товара

Для базы данных настроим связи между таблицами (рис. 1.1).

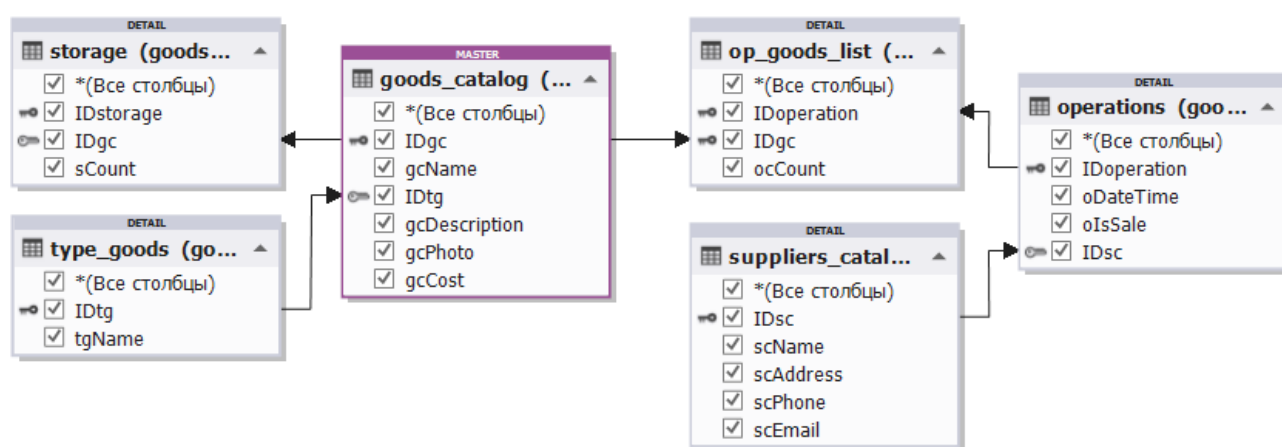


Рис. 1.1. Схема связей таблиц базы данных

Чтобы решить поставленную задачу, необходимо создать в СУБД **MySQL** базу данных. Перейдем к поэтапному рассмотрению процесса создания базы данных.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Система управления базами данных **MySQL** – это продукт развития СУБД **mSQL**, которую разработчики шведской компании **MySQL AB** взяли за основу при создании собственной базы данных.

Достоинствами СУБД **MySQL** являются:

- высокая скорость выполнения запросов. **MySQL** наряду с **Oracle** считается одной из самых быстрых СУБД в мире;
- использование при разработке широко распространенных языков программирования **C/C++**;
- функционирование более чем на 20 платформах, среди которых **Windows, Linux, FreeBSD, HP-UX, Mac OS X, OS/2, Solaris** и др.;
- открытый код, который доступен для просмотра и модернизации всем желающим. Общедоступная лицензия **GPL** (General Public License)

позволяет постоянно улучшать программный продукт и быстро находить и устранять уязвимые места. Особенностью лицензии **GPL** является тот факт, что любой код, скомпилированный с **GPL**-кодом, подпадает под **GPL**-лицензию, т. е. может свободно распространяться с предоставлением исходных кодов всем желающим;

- поддержка программного интерфейса приложения **API** (*Application Programming Interface* – программный интерфейс приложения) для огромного количества языков программирования;

- наличие встроенного сервера. СУБД **MySQL** может использоваться в качестве как внешнего сервера, поддерживающего соединение с локальным компьютером и с удаленным хостом, так и встроенного сервера. Достаточно скомпилировать программу с библиотекой встроенного сервера и приложение уже будет содержать полноценную СУБД **MySQL** с возможностью создания баз данных, таблиц и осуществления запросов к ним;

- широкий выбор подсистем хранения таблиц, в том числе и сторонних разработчиков, что позволяет реализовать оптимальные для решаемой задачи производительность и функциональность;

- корректно выполненная локализация. Например, у пользователя не возникнет проблем из-за того, что в конце работы над проектом поддержка базой данных сортировки русских строк окажется невозможной;

- начиная с версии 5.0, практически полное соответствие стандарту языка SQL и, следовательно, совместимость с другими базами данных.

Все эти особенности сделали **MySQL** стандартной базой данных, используемой на серверах хост-провайдеров и в качестве встроенной базы данных в прикладных программах.

Далее мы рассмотрим вопросы, касающиеся создания и администрирования базы данных в СУБД **MySQL**.

1.1 СОЗДАНИЕ БАЗЫ ДАННЫХ В ПРОГРАММЕ dbFORGE STUDIO FOR MySQL

Для создания новой пустой базы данных будем использовать визуальный редактор **dbForge Studio for MySQL** (существуют и другие клиентские приложения для работы с базами данных MySQL: *HeidiSQL*, *MySQL Workbench*, *DBeaver*, *PHPMyAdmin* и т.д. *dbForge Studio* используется так как является наиболее функциональной системой).

Базу данных для **MySQL** можно формировать в консольном режиме с использованием SQL-команд через клиента **mysql.exe**, который поставляется вместе с сервером, но такой способ является трудоемким, поскольку в нашем случае для создания базы данных, таблиц и связей между ними большое количество SQL-конструкций придется набирать вручную.

1.1.1 Подключение к серверу БД

Необходимо установить соединение с запущенным сервером **MySQL**, который может находиться как на локальном компьютере, так и удаленно.

Выберем пункт меню **База данных** и в открывшемся списке щелкнем на пункте **Новое соединение** (рис. 1.2). В качестве хоста зададим **localhost**. *MySQL* по умолчанию использует порт **3306**. Работа с базой данных будет осуществляться от имени суперпользователя. Для этого в поле **Имя** запишем **root**. Дадим название соединению и нажмем кнопку **Соединить**.

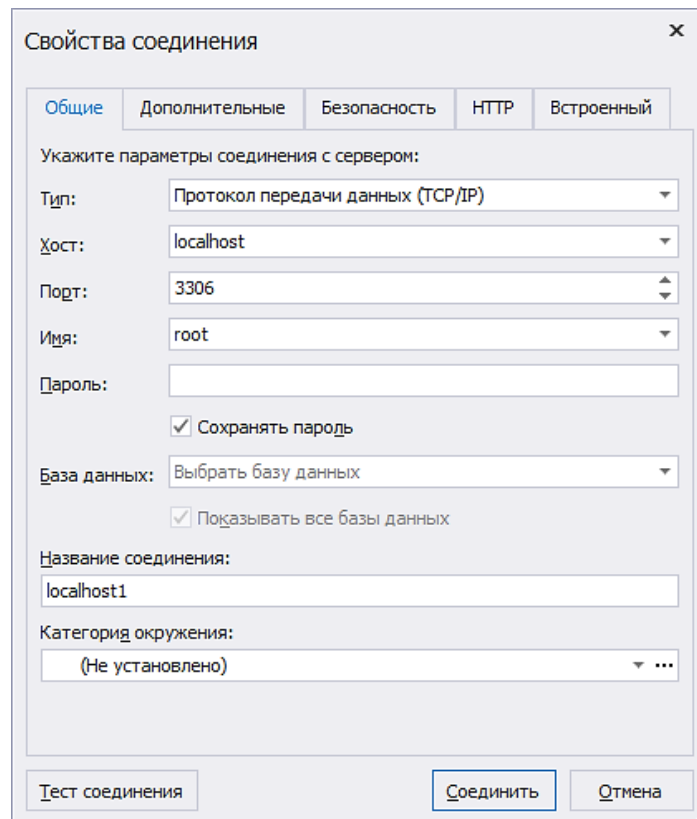


Рис. 1.2. Создание нового соединения

При соединении с сервером в области проводника будут отображены служебные базы данных **information_schema**, **mysql**, **performance_schema**, необходимые для функционирования и администрирования СУБД.

База данных **information_schema** обеспечивает доступ к метаданным базы данных и является набором доступных только для чтения представлений (**views**), которые содержат информацию обо всех таблицах, представлениях, столбцах и процедурах, хранящихся в БД.

Служебная база данных **mysql** является системным каталогом СУБД, создается во время установки **MySQL** и хранится на диске. Она содержит информацию, необходимую для правильного функционирования СУБД: о

поддерживаемых базах данных и их базовых таблицах, представлениях, курсорах, индексах, пользователях и их правах доступа к информации и т. д. Пользователь, имеющий права администратора, может модифицировать, добавлять и удалять данные из системного каталога.

База данных **performance_schema** предназначена для низкоуровневого мониторинга производительности сервера БД. Она позволяет пользователям контролировать наиболее ресурсоемкие запросы, объекты, приложения и учетные записи, обеспечивает доступ к сводкам с агрегированной статистикой.

1.1.2 Создание БД

Далее создадим собственную базу данных, выбрав пункт меню **База данных** и щелкнув на подпункте меню **Новая база данных** (рис. 1.3).

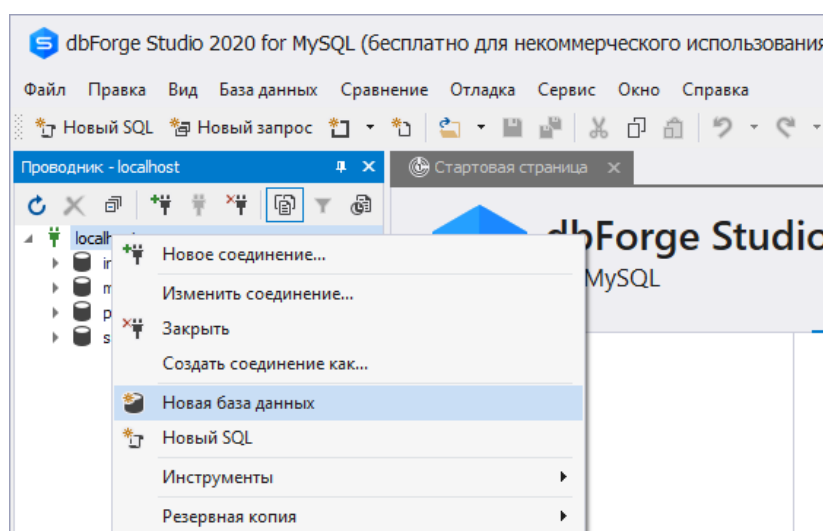


Рис. 1.3. Пример создания БД

Введем имя базы данных **GoodsManager** (в нашем примере БД будет осуществлять хранение информации магазина компьютерной техники) выберем кодировку **utf8mb4** (*UTF-8 Multibyte 4*) и нажмем кнопку применить изменения (рис. 1.4). Кодировка – это таблица сопоставлений символов, которые мы можем видеть на экране, определенным числовым кодам. Каждый символ, который мы вводим с клавиатуры, либо видим на экране монитора, закодирован определенной последовательностью битов (нулей и единиц). Внешний вид самих символов определяется файлами шрифтов, которые установлены на компьютере.

В контексте разработки, можно столкнуться с некорректным отображением символов, когда, к примеру, русский текст в базе хранится в одной кодировке (*UTF-8*), а в разрабатываемом приложении по ошибке отображается с помощью другой (*Windows-1251*).

Решить эту проблему позволяет применение универсальной кодировки **UTF-8** (*Unicode Transformation Format*) – это многобайтовая кодировка с переменной длиной символа. В нашей базе мы применяем расширенный вариант **utf8mb4**, которая позволяет кодировать дополнительные символы, например, *Emoji*.

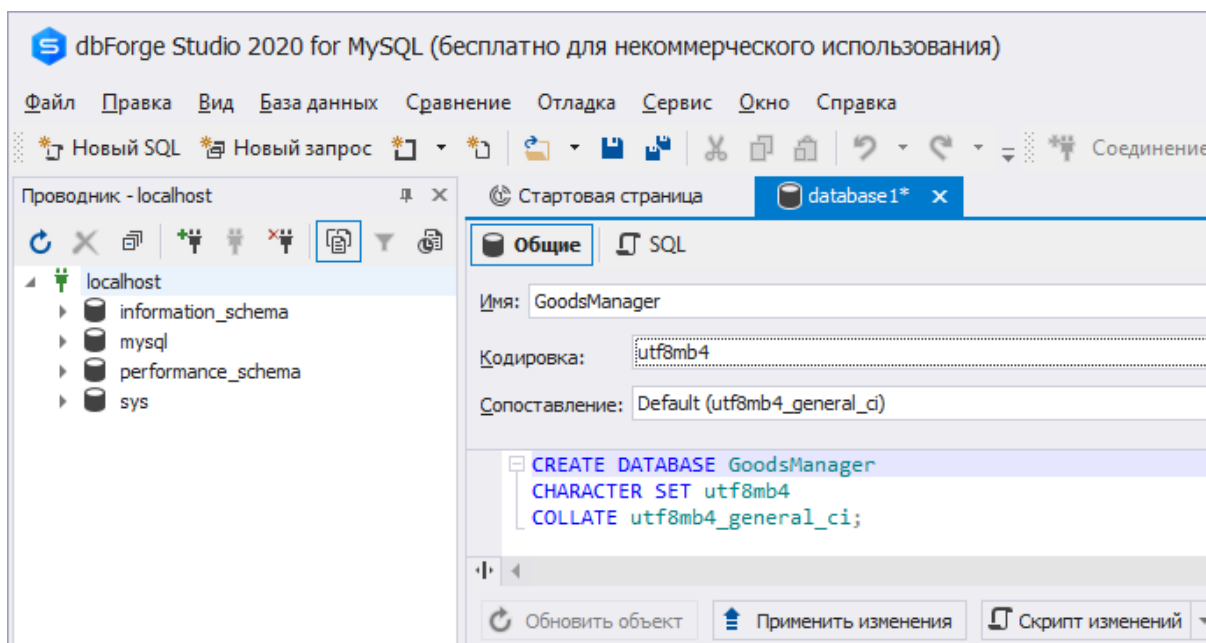


Рис. 1.4. Пример настройки БД

Так как мы работаем с серверной системой, которая обычно находится на другом компьютере, необходимо обновлять информацию после выполнения каких-либо действий с БД (все запросы выполняются на сервере, и чтобы увидеть применяемые изменения в приложении, необходимо чтобы сервер отправил нам их) (рис. 1.5).

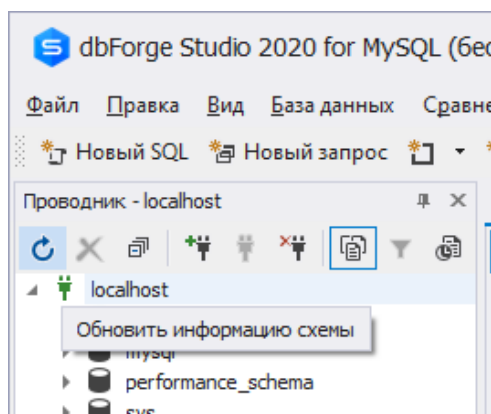


Рис. 1.5. Пример обновления данных

Теперь перейдем к рассмотрению процесса создания таблиц.

1.2 СОЗДАНИЕ ТАБЛИЦ

MySQL поддерживает различные подсистемы хранения таблиц (*Engines*):

- **InnoDB** наиболее широко используемый движок с поддержкой транзакций (*ACID*). Основное отличие от других, поддержка ссылочной целостности с внешними ключами.

- **MyISAM** применяется для не транзакционных таблиц с блокировками на уровне таблицы и полнотекстовым поиском для индексов. В основном используется в *web*.

- **Federated** предоставляет возможность объединять различные *MySQL*-серверы для создания одной логической базы данных из нескольких физических компьютеров.

- **Memory** создает таблицы и хранит их в оперативной памяти для повышения быстродействия и ускорения доступа к данным.

- **Merge** используется для логического объединения одинаковых *MyISAM*-таблиц в одну. Может использоваться для обработки больших объемов данных.

- **Archive** используется для хранения больших объемов неиндексированных данных.

- **CSV** гибкое хранилище для *CSV (Comma-Separated Values)* файлов. Используется для экспорта/импорта данных в этом формате.

- **Blackhole** принимает данные, но ничего не возвращает.

Для создания таблиц выберем базу данных в проводнике, щелкнем правой кнопкой мыши на подкаталоге **Таблицы** и в появившемся контекстном меню выберем пункт **Новая таблица** (рис. 1.6).

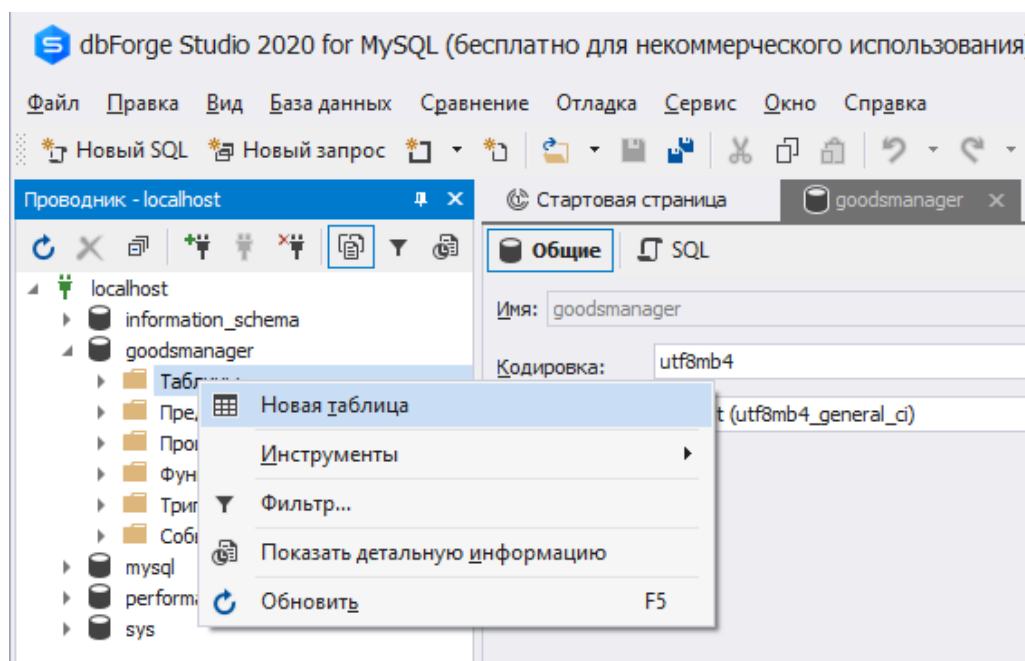


Рис. 1.6. Создание таблицы

Введем имя таблицы **type_goods** (Типы товаров, определяет категории товаров), укажем подсистему хранения (**Движок**, *Engine*), как **InnoDB**. Создадим новый столбец **IDtg** (Код типа товаров – идентификатор, по которому можно ссылаться на конкретную категорию товаров) (рис. 1.7).

Укажем, что **IDtg** является ключевым (**PRIMARY KEY**) полем установив флажок напротив имени столбца.

Для каждого создаваемого поля таблицы выбирается его тип и длина (точность и масштаб). Точность (*Precision*) представляет собой общее количество цифр в числе, а масштаб (*Scale*) задает количество десятичных знаков после запятой. Например, число **123.45** имеет точность **5** и масштаб **2**.

С помощью флажков выставляются свойства полей: без знака, автоинкремент (для ключей), пустые значения и др.

Атрибут **Автоинкремент** (**AUTO_INCREMENT**) используется для генерации уникальных идентификаторов для новых строк таблицы.

Атрибут **Значение по умолчанию** позволяет задать значения которые автоматически будут заноситься в запись при её создании. При создании поля данный атрибут сразу принимает значение NULL (т.е. отсутствие значения), однако в зависимости от логики работы базы данных можно установить желаемое значение по умолчанию.

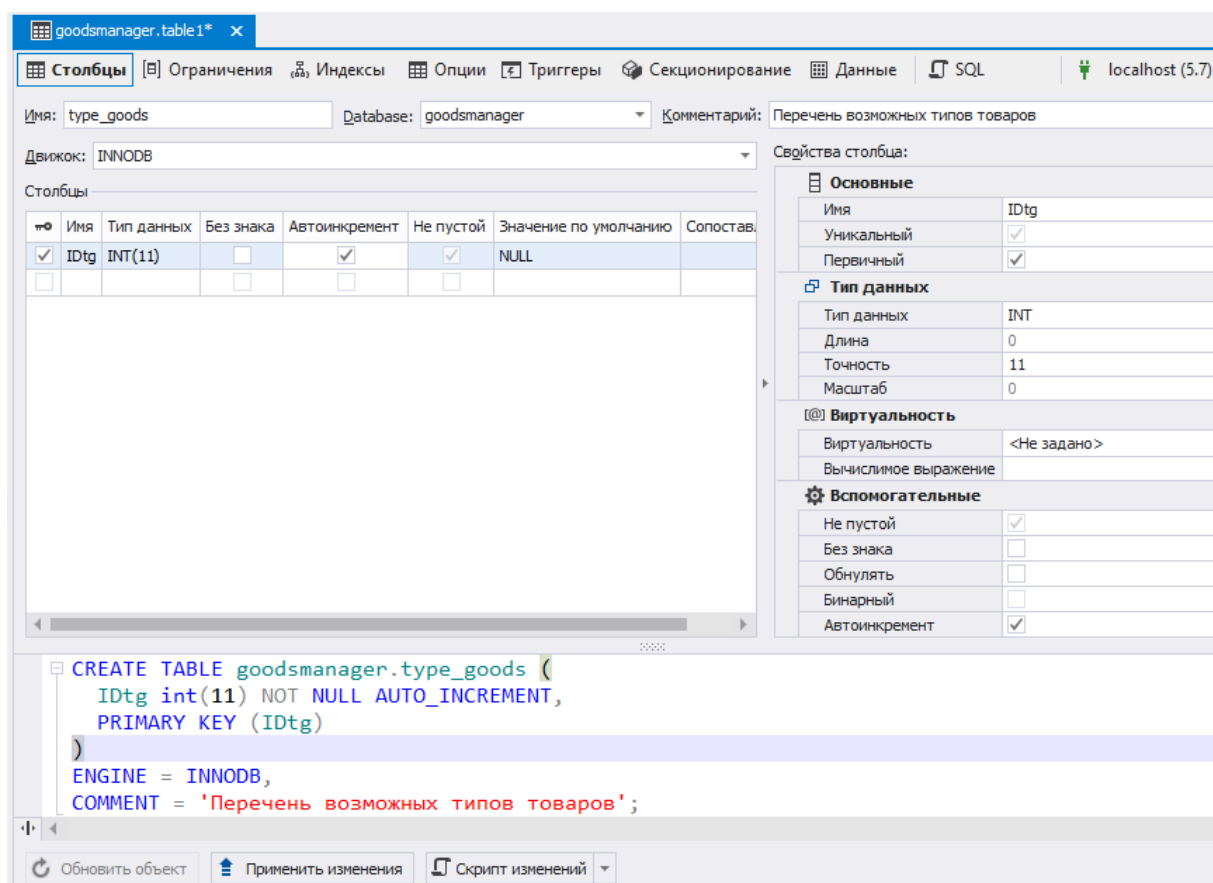


Рис. 1.7. Создание столбцов таблицы

Комментарии (COMMENT) к создаваемым таблицам и столбцам не обязательны, но вводятся для того, чтобы разработчикам БД в будущем было легче сориентироваться в предметной области.

Система управления базами данных **MySQL** поддерживает типы данных, которые можно разделить на основные категории: строковые данные, числовые данные, дата и время, описание которых представлено в табл. 1.2. У каждого типа есть своя длина – количество знакомест, в которых будет размещено все число при его передаче.

Таблица 1.2

Типы данных СУБД MySQL

Тип данных	Описание
Строковые типы данных	
CHAR(size)	Строка фиксированной длины (может содержать буквы, числа, специальные символы), <i>size</i> - количество символов 0 до 255
VARCHAR(size)	Строка переменной длины (может содержать буквы, числа, специальные символы), <i>size</i> - количество символов 0 до 65535
BINARY(size)	Соответствует CHAR, но хранит строки в бинарном виде. <i>size</i> – длина строки в байтах
VARBINARY(size)	Соответствует VARCHAR, но хранит строки в бинарном виде, <i>size</i> – длина строки в байтах
TINYBLOB	Для бинарных объектов (Binary Large Objects). Максимальная длина: 255 байт
TINYTEXT	Хранит строки длиной до 255 символов
TEXT(size)	Хранит строки длиной до 65,535 байт
BLOB(size)	Для бинарных объектов. Хранит до 65,535 байт данных
MEDIUMTEXT	Хранит строки длиной до 16,777,215 символов
MEDIUMBLOB	Для бинарных объектов. Хранит до 16,777,215 байт данных
LONGTEXT	Хранит строки длиной до 4,294,967,295 символов
LOBLOB	Для бинарных объектов. Хранит до 4,294,967,295 байт данных
ENUM(v1, v2, v3, ...)	Строковый объект который имеет только одно значение, выбираемое из списка возможных значений. Список может содержать 65535 значений.
SET(v1, v2, v3, ...)	Строковый объект который имеет 0 или более значений, выбираемое из списка возможных значений. Список может содержать 64 значения
Числовые типы данных	
BIT(size)	Битовое значение. Количество бит на хранимое данное, которое задается <i>size</i> и может быть от 1 до 64
TINYINT(size)	Маленькое целое число в диапазоне от -128 до 127, если со знаком. Без знака от 0 до 255, <i>size</i> количество отображаемых значений (до 255)
BOOL	Ноль считается как false, остальные значения как true
BOOLEAN	Соответствует BOOL
SMALLINT(size)	Небольшое целое число в диапазоне от -32768 до 32767, если со знаком. Без знака от 0 до 65535, <i>size</i> количество отображаемых значений (до 255)

Тип данных	Описание
MEDIUMINT(<i>size</i>)	Среднее целое число в диапазоне от -8388608 до 8388607, если со знаком. Без знака от 0 to 16777215, <i>size</i> количество отображаемых значений (до 255)
INT(<i>size</i>)	Целое число в диапазоне от -2147483648 до 2147483647, если со знаком. Без знака от 0 до 4294967295
INTEGER(<i>size</i>)	Соответствует INT
BIGINT(<i>size</i>)	Большое целое число в диапазоне от -223372036854775808 до 9223372036854775807, если со знаком. Без знака от 0 до 18446744073709551615
FLOAT(<i>size</i> , <i>d</i>)	Число с плавающей точкой. Общее количество цифр определяется <i>size</i> . Количество цифр после запятой определяется <i>d</i> .
DOUBLE (<i>size</i> , <i>d</i>)	Число с плавающей точкой двойной точности. Общее количество цифр определяется <i>size</i> . Количество цифр после запятой определяется <i>d</i> .
DECIMAL(<i>size</i> , <i>d</i>)	Число с фиксированной точкой. Общее количество цифр определяется <i>size</i> . Количество цифр после запятой определяется <i>d</i> . Максимум для <i>size</i> - 65. Максимум для <i>d</i> - 30. По умолчанию <i>size</i> равен 10, а <i>d</i> равно 0.
DEC(<i>size</i> , <i>d</i>)	Соответствует to DECIMAL
Типы данных для хранения даты и времени	
DATE	Дата в формате: YYYY-MM-DD. Поддерживает диапазон дат от '1000-01-01' до '9999-12-31'
DATETIME(<i>fsp</i>)	Комбинация даты и времени в формате: YYYY-MM-DD hh:mm:ss. Поддерживает диапазон дат от '1000-01-01 00:00:00' до '9999-12-31 23:59:59'. По умолчанию инициализируется текущей датой и временем.
TIMESTAMP(<i>fsp</i>)	Временная метка. Значения хранятся как количество секунд, прошедшее с '1970-01-01 00:00:00' UTC (Unix epoch). Формат: YYYY-MM-DD hh:mm:ss. Поддерживает диапазон дат от '1970-01-01 00:00:01' UTC до '2038-01-09 03:14:07' UTC.
TIME(<i>fsp</i>)	Время в формате: hh:mm:ss. Поддерживает диапазон от '-838:59:59' до '838:59:59'
YEAR	Год в формате четырех цифр. Доступные значения: от 1901 до 2155, и 0000

Далее создадим новый столбец **tgName** (Наименование типа товаров), установим для него параметры. Для сохранения таблицы в базе данных нажмем кнопку **Применить изменения** (рис. 1.8) или воспользовавшись контекстным меню названия вкладки выберем пункт **Сохранить goodsmanager.table1**. После сохранения таблица будет иметь заданное имя.

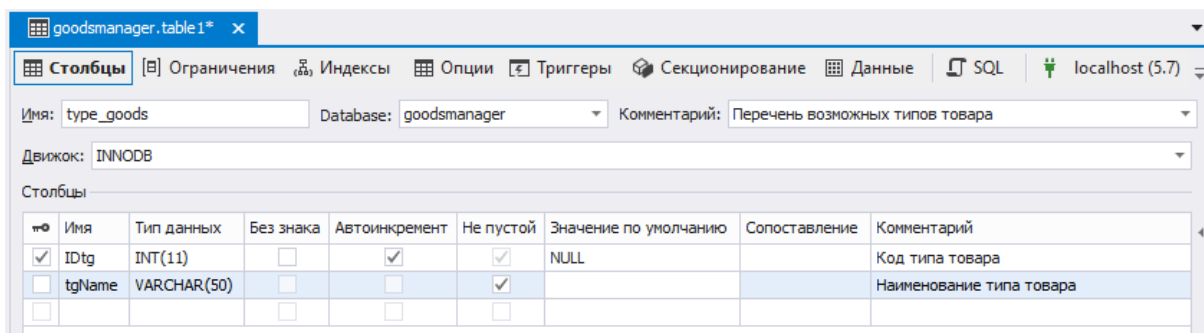


Рис. 1.8. Таблица *type_goods* после создания столбца *tgName* и применения изменений.

Создадим еще одну таблицу **goods_catalog** для хранения конкретных товаров определенного типа (категории). Заполним параметры столбцов как показано на рис. 1.9 и сохраним изменения.

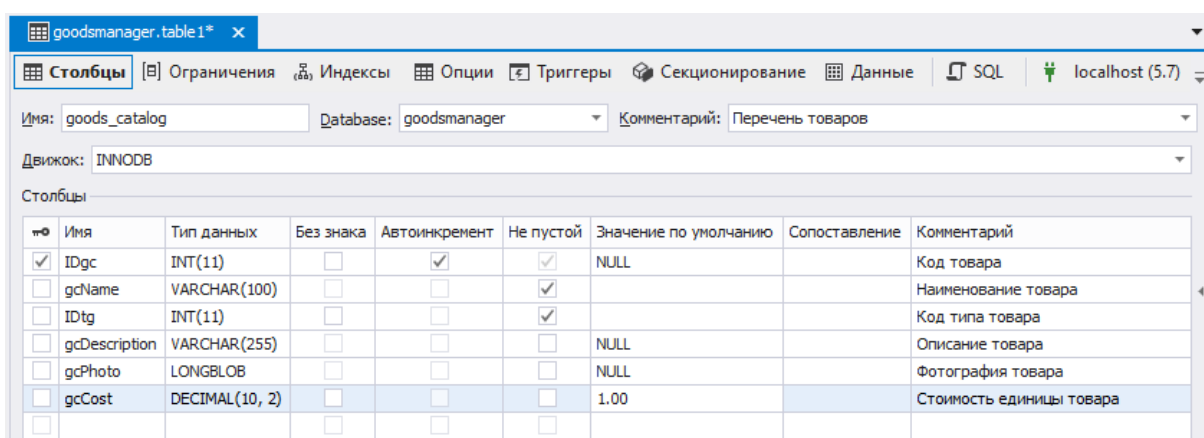


Рис. 1.9. Создание таблицы *goods_catalog*

Для хранения данных о поставщиках товаров создадим таблицу **suppliers_catalog** (рис. 1.10).

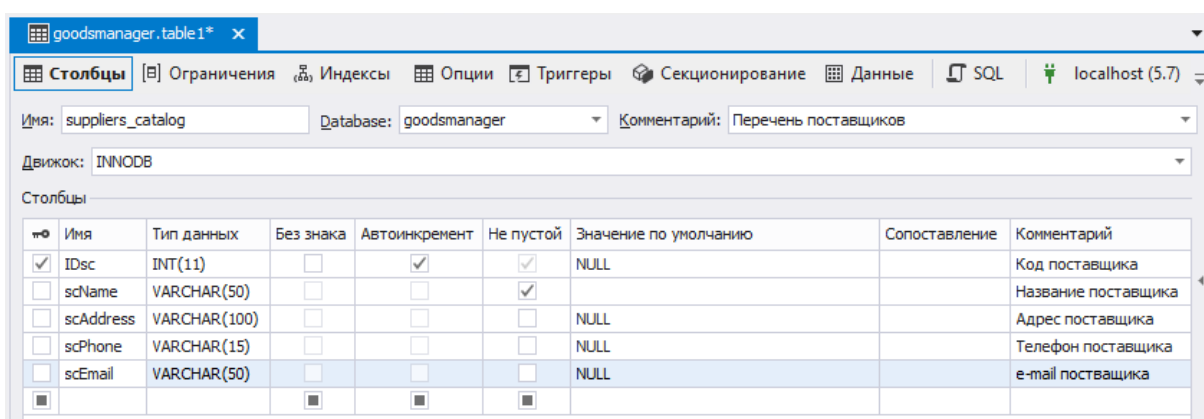


Рис. 1.10. Создание таблицы *suppliers_catalog*

Далее создадим таблицу **operations** отвечающую за операции с товарами (приход или продажа) (рис. 1.11),

goodsmanager.table1* x

Столбцы [B] Ограничения Индексы Опции Триггеры Секционирование Данные SQL localhost (5.7)

Имя: operations Database: goodsmanager Комментарий: Операции с товарами (приход/продажа)

Движок: INNODB

Столбцы

#	Имя	Тип данных	Без знака	Автоинкремент	Не пустой	Значение по умолчанию	Сопоставление	Комментарий
☒	IDoperation	INT(11)	☐	☒	☒	NULL		Код операции
☐	oDateTime	DATETIME	☐	☐	☒			Дата операции
☐	oIsSale	TINYINT(1)	☐	☐	☐	1		Тип операции (1-продажа 0-поставка)
☐	IDsc	INT(11)	☐	☐	☐	NULL		Код поставщика

Рис. 1.11. Создание таблицы **operations**

Теперь необходимо создать вспомогательную таблицу **op_goods_list**, которая будет реализовывать связь многие-ко-многим между товарами и операциями, которые можно с ними совершить (рис. 1.12).

goodsmanager.table1* x

Столбцы [B] Ограничения Индексы Опции Триггеры Секционирование Данные SQL localhost (5.7)

Имя: op_goods_list Database: goodsmanager Комментарий: Состав товаров в операции

Движок: INNODB

Столбцы

#	Имя	Тип данных	Без знака	Автоинкремент	Не пустой	Значение по умолчанию	Сопоставление	Комментарий
☒	IDoperation	INT(11)	☐	☐	☒	NULL		Код операции
☒	IDgc	INT(11)	☐	☐	☒			Код товара
☐	ocCount	INT(11)	☐	☐	☒	1		Количество товара

Рис. 1.12. Создание таблицы **op_goods_list**

Обратите внимание на составной первичный ключ состоящий из полей **IDoperations** и **IDgc**, более того эти поля будут использоваться для создания связей с таблицами **operations** и **goods_catalog**.

Теперь создадим дополнительную и последнюю таблицу **storage**, описывающую складские запасы магазина и необходимую в дальнейшем для демонстрации функциональных возможностей СУБД (рис. 1.13).

goodsmanager.table1* x

Столбцы [B] Ограничения Индексы Опции Триггеры Секционирование Данные SQL localhost (5.7)

Имя: storage Database: goodsmanager Комментарий: Складские запасы

Движок: INNODB

Столбцы

#	Имя	Тип данных	Без знака	Автоинкремент	Не пустой	Значение по умолчанию	Сопоставление	Комментарий
☒	IDstorage	INT(11)	☐	☒	☒	NULL		Код складской записи
☐	IDgc	INT(11)	☐	☐	☒			Код товара
☐	sCount	INT(11)	☐	☐	☒	0		Количество товара

Рис. 1.13. Создание таблицы **storage**

После создания таблиц можно перейти к установке связей между таблицами базы данных. Связи позволяют обеспечить ссылочную целостность данных, тем самым ограничивая набор возможных значений при добавлении.

1.3 УСТАНОВКА СВЯЗЕЙ

Для начала рассмотрим пример обеспечения ссылочной целостности данных создав связь между типом товара и самим товаром. Для этого в открытой на редактировании таблице *goods_catalog* в разделе **Ограничения** добавим внешний ключ (*FOREIGN KEY*) кликнув правой кнопкой мыши на пустом месте и выбрав пункт контекстного меню **Создать внешний ключ** (рис. 1.14).

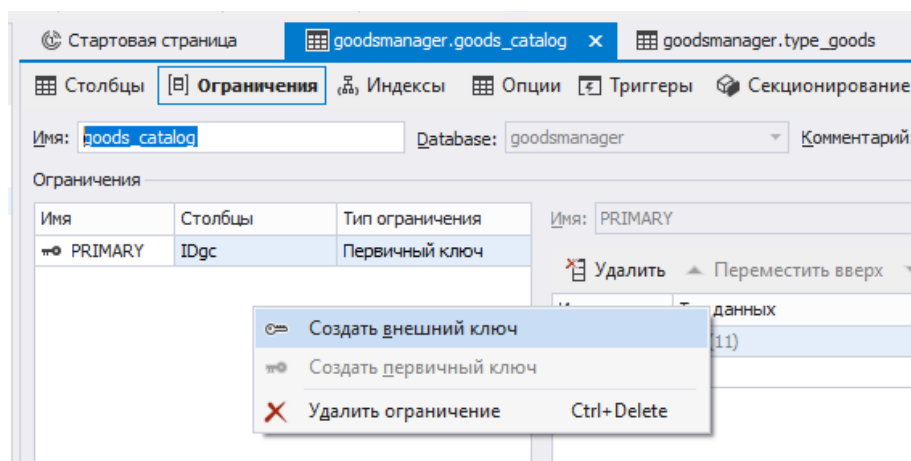


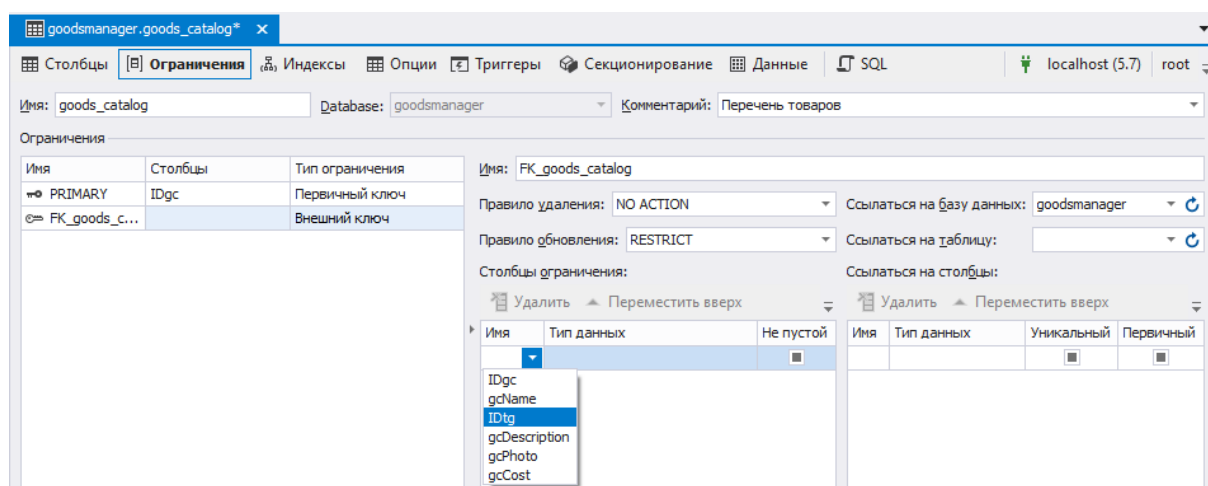
Рис. 1.14. Контекстное меню создания внешнего ключа

В нашем случае внешний ключ должен связывать поля **IDtg** в обеих таблицах, являющихся идентификаторами категории товаров (рис. 1.15). Внешние ключи создаются для **дочерних** таблиц. Дочерними считаются таблицы которые зависят от других таблиц, например, товары из *goods_catalog* ссылаются на категории товаров из *type_goods*.

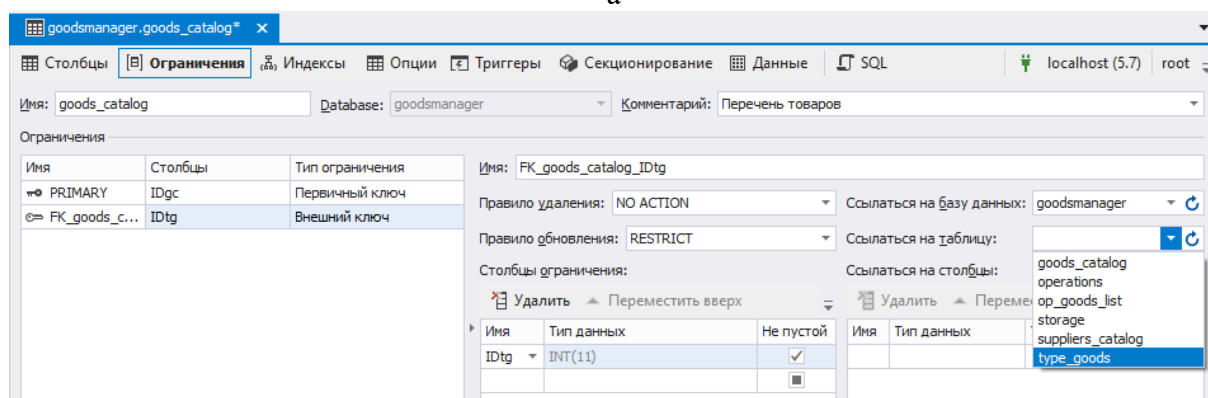
Внешний ключ – это столбец или сочетание столбцов, которое применяется для принудительного установления связи между данными в двух таблицах. Для внешнего ключа можно настроить правила обновления и удаления данных:

- **CASCADE** – удаление или изменение записи, содержащей первичный ключ, приведет к автоматическому удалению или изменению соответствующих записей в дочерней таблице. Например, удаляя категорию **Ноутбуки** в *type_goods* также удалятся все товары из *goods_catalog* относящиеся к этой категории;

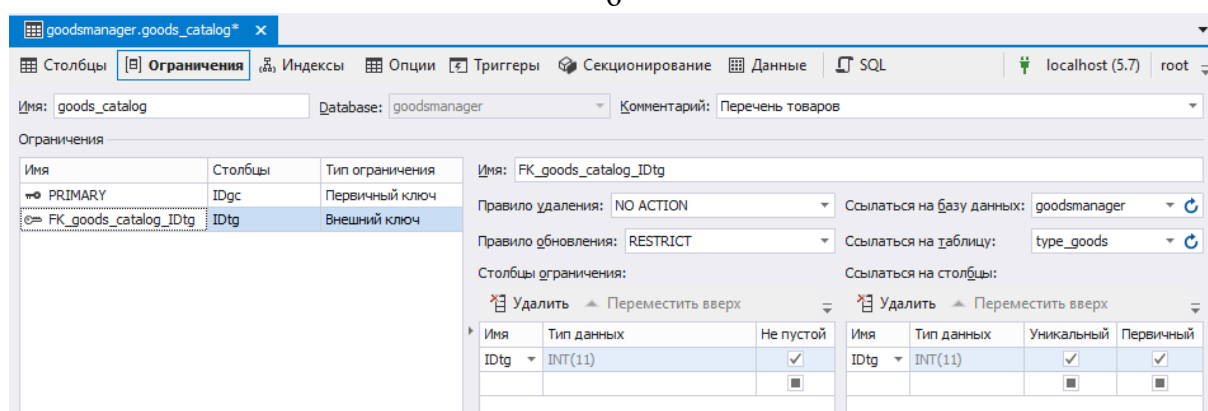
- **SET NULL** – при удалении или изменении записи, содержащей первичный ключ, соответствующие записи в дочерней таблице получают значение **NULL**;
- **RESTRICT** – нельзя удалить или изменить запись, пока в дочерней таблице существуют ссылающиеся записи;
- **NO ACTION** – в *MySQL* это правило работает аналогично *RESTRICT*.



а



б



в

Рис. 1.15. Этапы создания внешнего ключа: а – выбор столбца, задающего ограничение; б – выбор таблицы на которую будет ссылаться внешний ключ; в – выбор столбца для связи.

Настройка ограничений при помощи внешних ключей необходима для таблиц *operations*, *op_goods_list* и *storage*. Пример настроенного внешнего ключа таблицы *operations* приведен на рис. 1.16.

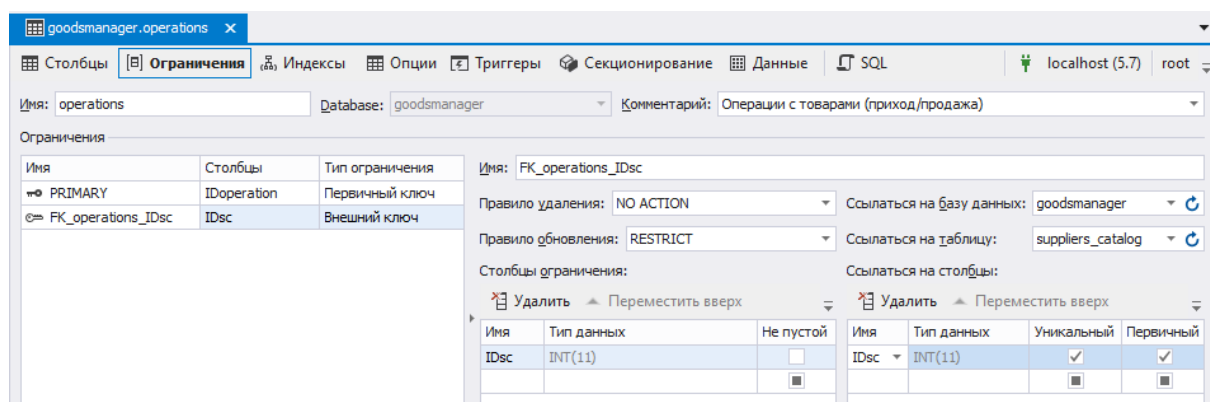


Рис. 1.16. Параметры внешнего ключа таблицы *operations*

На рисунке 1.17 приведены параметры для двух внешних ключей таблицы *op_goods_list* отвечающие за связи с таблицами **Операции с товарами** и **Товары**.

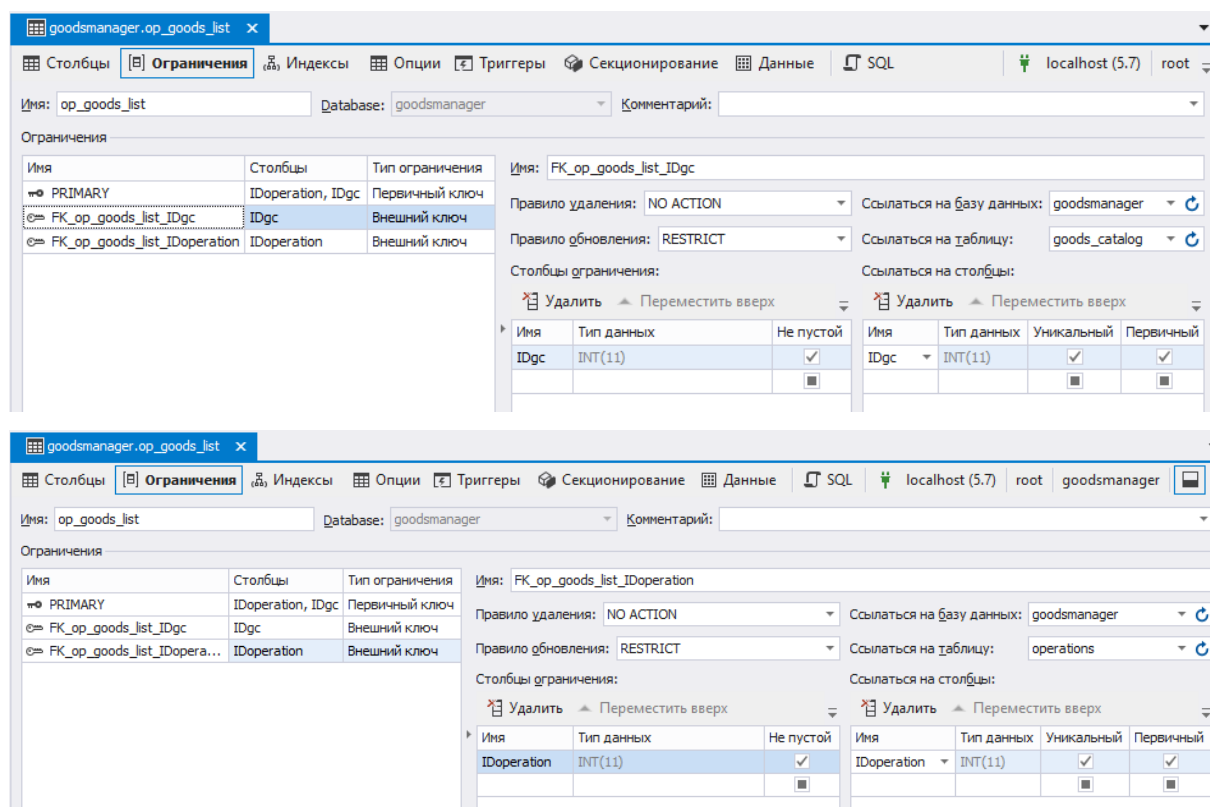


Рис. 1.17. Параметры внешних ключей таблицы *op_goods_list*

Таблица **Склад товаров** имеет в качестве ограничения один внешний ключ по полю **IDgc** (рис. 1.18)

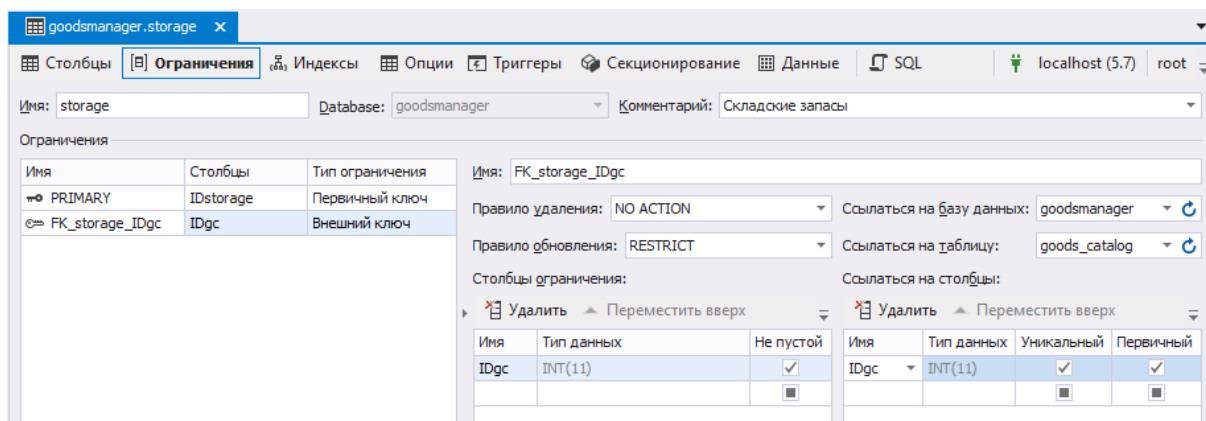


Рис. 1.18. Параметры внешнего ключа таблицы *storage*

После создания таблиц можно перейти к рассмотрению способов определения и настройки индексов.

1.4 НАСТРОЙКА ИНДЕКСОВ

Индексы — это вспомогательные объекты БД предназначенные для повышения производительности поиска и сортировки записей. Индекс создается для одного или нескольких полей (составной индекс). Сам индекс может быть гораздо меньше описываемой им таблицы (его размер зависит от количества уникальных значений проиндексированных полей), делая чтение данных более эффективным. Без индекса чтение данных осуществляется по всей таблице, начиная с первой записи, пока не будут найдены соответствующие строки. В этом случае, чем больше таблица, тем больше будут накладные расходы.

Приступим к созданию индексов для таблицы **type_goods**. Создадим уникальный индекс для столбца **tgName**. Необходимость в уникальном индексе определяется предметной областью (уникальность индекса будет контролировать, чтобы вводимые данные не повторялись) Так, в нашем случае, тип товара не должен повторяться, это означает, что необходимо создать уникальный индекс.

Для создания индекса нужно перейти в раздел **Индексы (Indexes)**. Щелкнув правой кнопкой на пустую область выбрать пункт контекстного меню **Добавить уникальный индекс** (рис. 1.19). В качестве альтернативного способа можно нажать на кнопку **Добавить уникальный индекс**

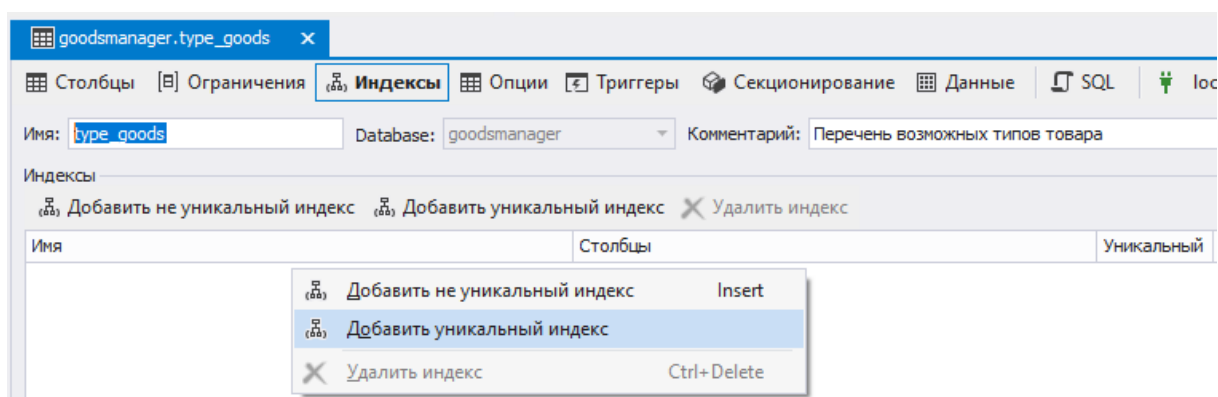


Рис. 1.19. Выбор создания индекса

Далее необходимо выбрать столбцы для индексирования – в нашем случае это столбец **tgName** (рис. 1.20). После этого можно применить изменения.

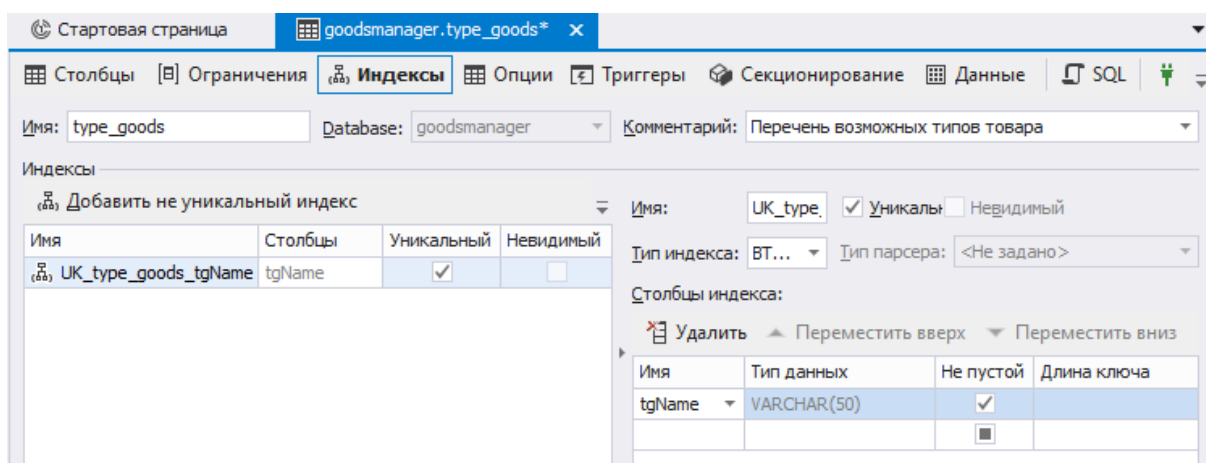


Рис. 1.20. Настройка уникального индекса таблицы *type_goods*

По умолчанию используется тип индекса **В-дерево** (*B-tree*), который покрывает широкий круг задач. С помощью В-дерева можно проиндексировать любые данные, которые могут быть отсортированы, т. е. для которых применимы операции сравнения больше/меньше/равно (числа, строки, дата и время, логический тип и любые данные, которые можно ими закодировать).

Индексы важный инструмент баз данных, ускоряющий поиск, но не стоит создавать много индексов без необходимости, так как они занимают дополнительную память, и при любом обновлении проиндексированных данных СУБД необходимо перестроить и сам индекс.

В случае таблицы **goods_catalog** добавим несколько индексов. При переходе на вкладку **Индексы** можно заметить, что для внешних ключей индексы были созданы автоматически. Создадим уникальный индекс для столбца **gcName** (рис 1.21).

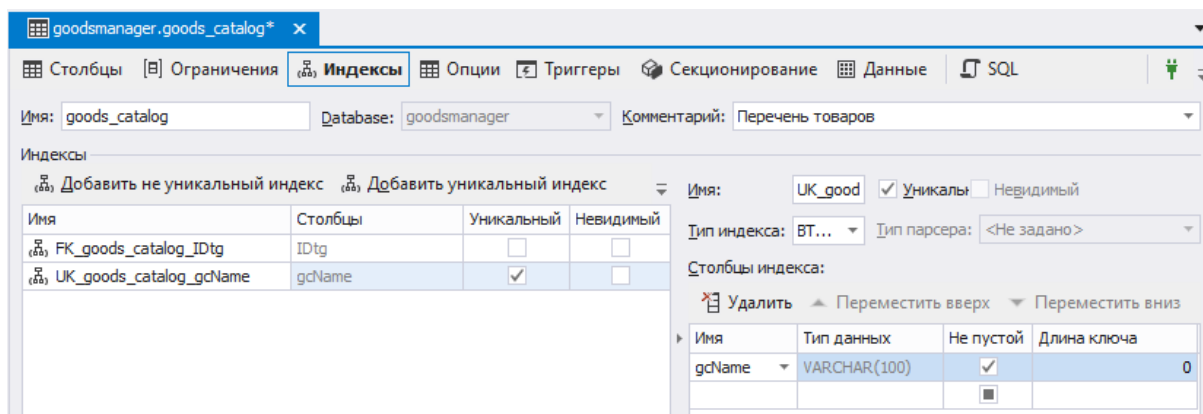
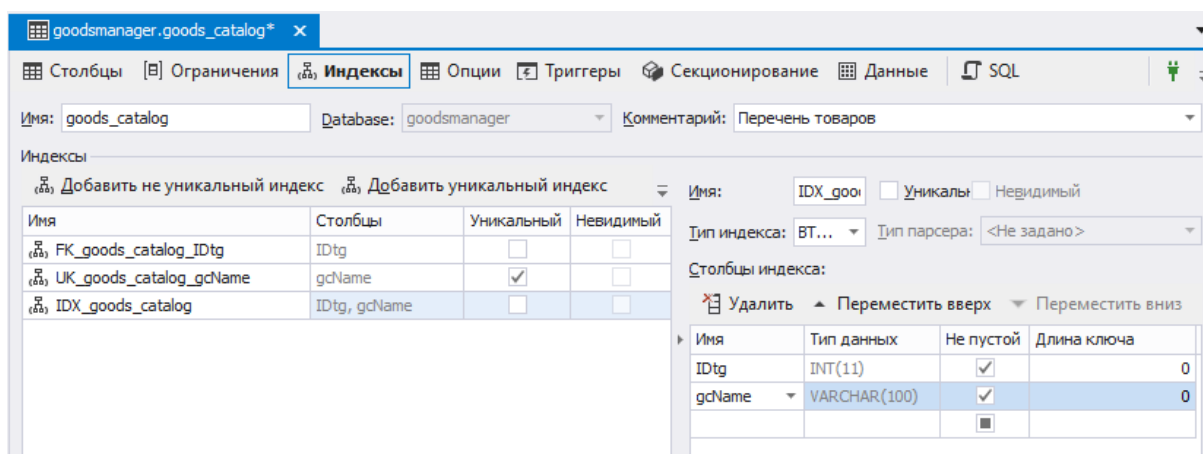
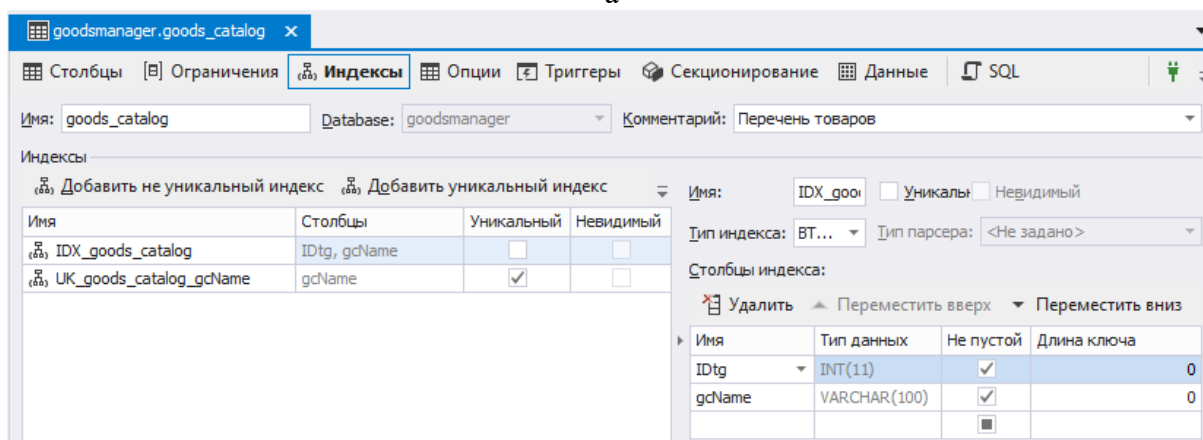


Рис. 1.21. Параметры уникального индекса поля *gcName*.

Теперь настроим второй индекс, он будет не уникальным и составным, созданный для полей **IDtg** и **gcName** (рис. 1.22 а). После настройки индекса и сохранения изменений таблицы, индекс для внешнего ключа исчезнет из перечня индексов (рис. 1.22 б), это обусловлено тем, что в индексе для внешнего ключа и в составном индексе задействовано одно и тоже поле **IDtg**.



а



б

Рис. 1.22. Параметры составного индекса в таблице *goods_catalog*: а – вид окна до сохранения; б – вид окна после сохранения изменений.

Для того чтобы быстро проводить выборку по дате в таблице *operations* добавим не уникальный индекс для поля **oDateTime** (рис 1.23).

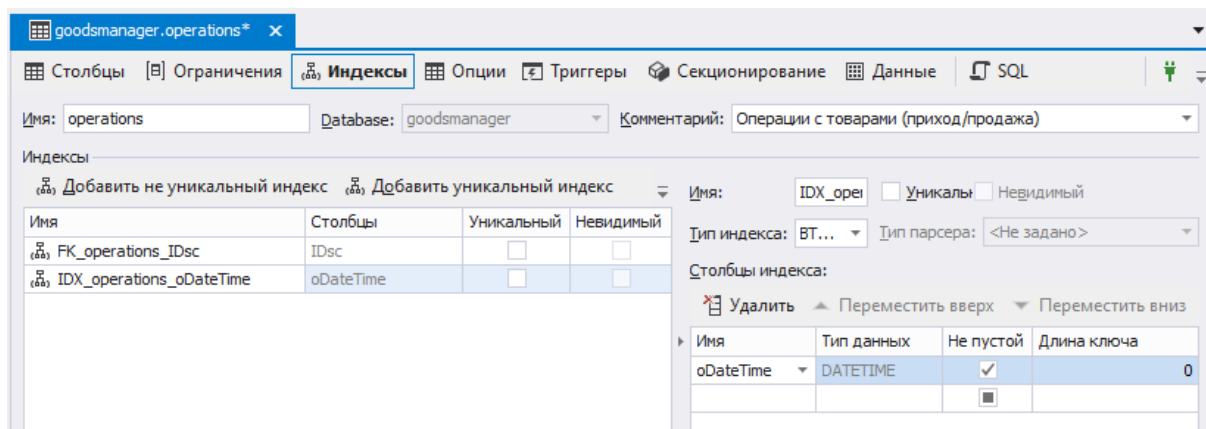


Рис. 1.23. Параметры индексов таблицы *operations*

Создадим уникальные индексы в таблице *suppliers_catalog* (Поставщики) по полям **scName** и **scEmail** (рис 1.24).

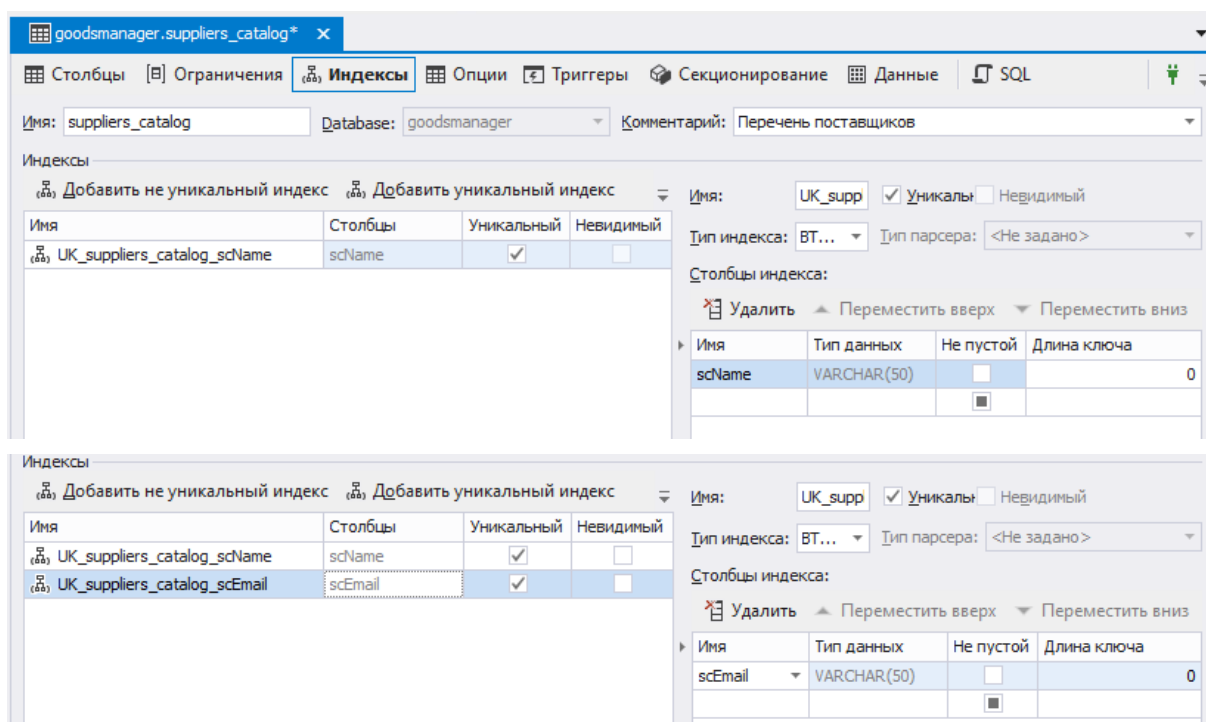


Рис. 1.24. Параметры индексов таблицы *suppliers_catalog*

Для таблиц *storage* (Склад товаров) и *op_goods_list* (Состав операций) индексы создались автоматически во время создания внешних ключей.

На этом этапе создание основных таблиц базы данных и их настройка завершена. В дальнейшем в БД будут добавляться различные динамические элементы (представления, запросы, функции, триггеры и др.).

Теперь необходимо проверить корректно ли были созданы связи и индексы. Для этого в **dbForge Studio for MySQL** выберем в основном меню пункт **База данных** и щелкнем по элементу **Диаграмма БД**. Перенесите одновременно все таблицы БД на открывшуюся область. Сгенерированная схема должна соответствовать физической диаграмме, представленной на рис. 1.25.

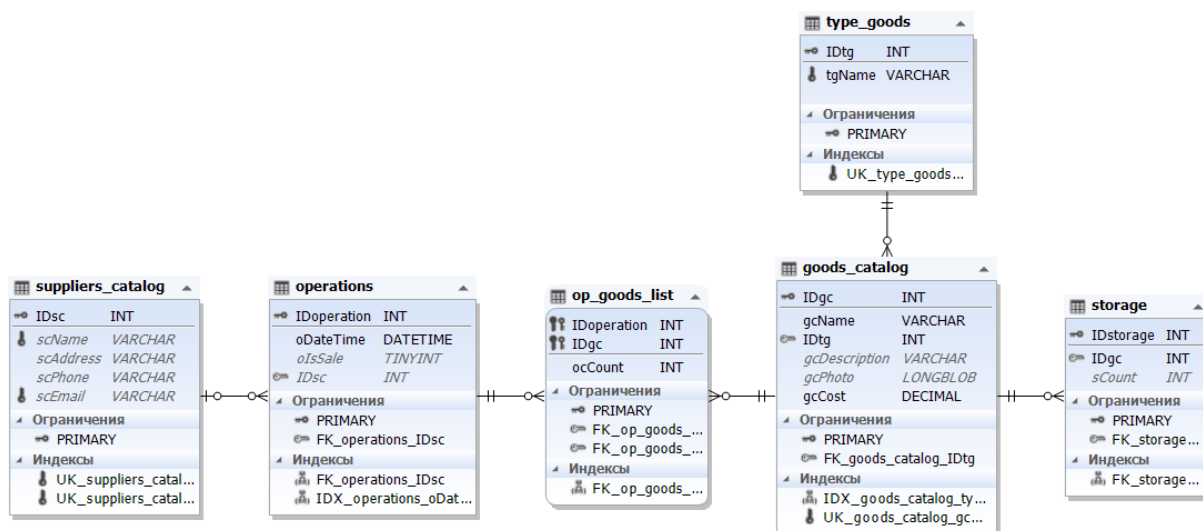


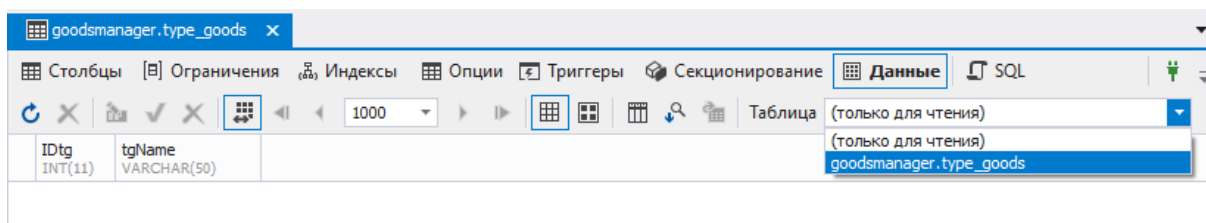
Рис. 1.25. Физическая схема базы данных

При этом следует понимать, что компоновка элементов на диаграмме по умолчанию может отличаться, а также будут закрыты все блоки **Ограничения** и **Индексы**. При желании компоновку можно сделать как на представленной схеме.

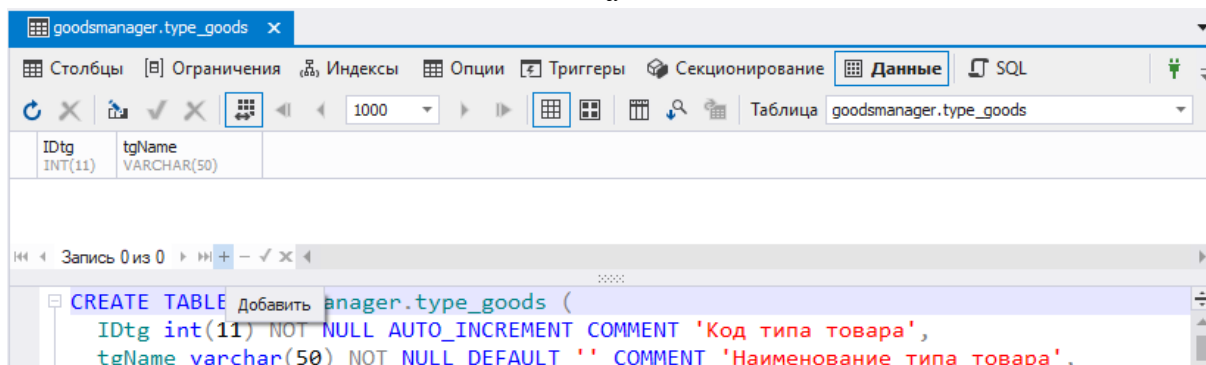
После проверки настройки связей таблиц базы данных и корректности установки индексации полей можно приступить к заполнению таблиц информацией.

1.5 ЗАПОЛНЕНИЕ ТАБЛИЦ ДАННЫМИ

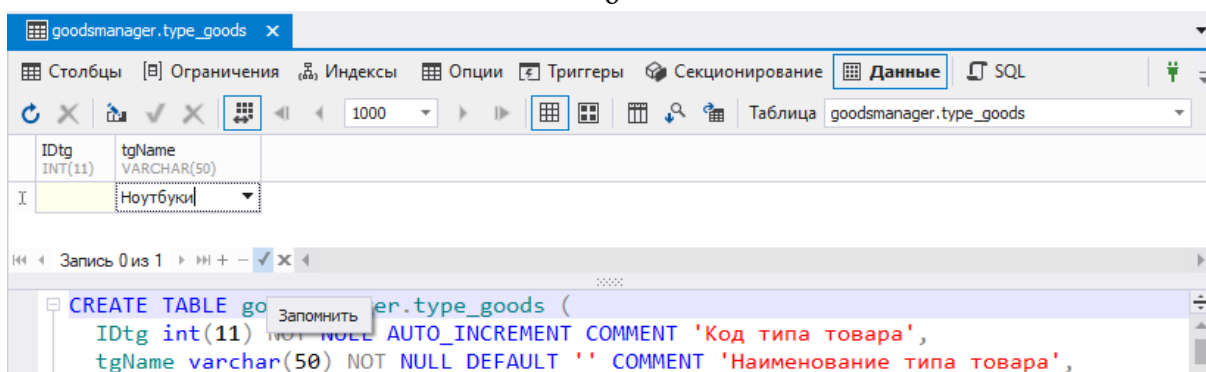
Рассмотрим пример заполнения данными сначала на таблице *type_goods*. Для того чтобы добавить записи в таблицу необходимо перейти в раздел **Данные**. По умолчанию показ данных происходит в режиме чтения, для того чтобы можно было изменять данные нужно в блоке **Таблица** вместо опции (только для чтения) нужно выбрать необходимую таблицу (рис. 1.26 а). После этого таблица станет доступна для редактирования информации. Чтобы добавить новую запись можно нажать клавишу **Insert** или кнопку со знаком + (означает добавить) (рис. 1.26 б). После ввода информации можно сохранить запись перейдя на другую строчку или нажать на кнопку со знаком ✓ (**Запомнить**) (рис. 1.26 в).



а



б



в

Рис. 1.26. Этапы добавления записи: а – перевод в режим редактирования данных; б – выбор добавления записи; в – заполнение данными и выбор запоминания записи.

Далее, в базу данных магазина компьютерной техники, занесем информацию о трех типах (категориях) товаров. Пример внесенных данных показан на рис. 1.27.

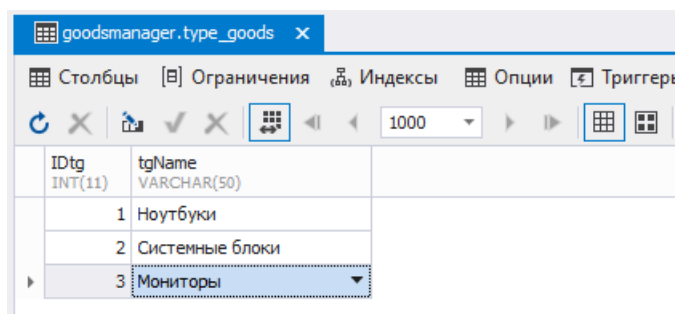


Рис. 1.27. Пример заполнения таблицы *type_goods*

Теперь добавим записи в таблицу **goods_catalog** данными (рис. 1.28).

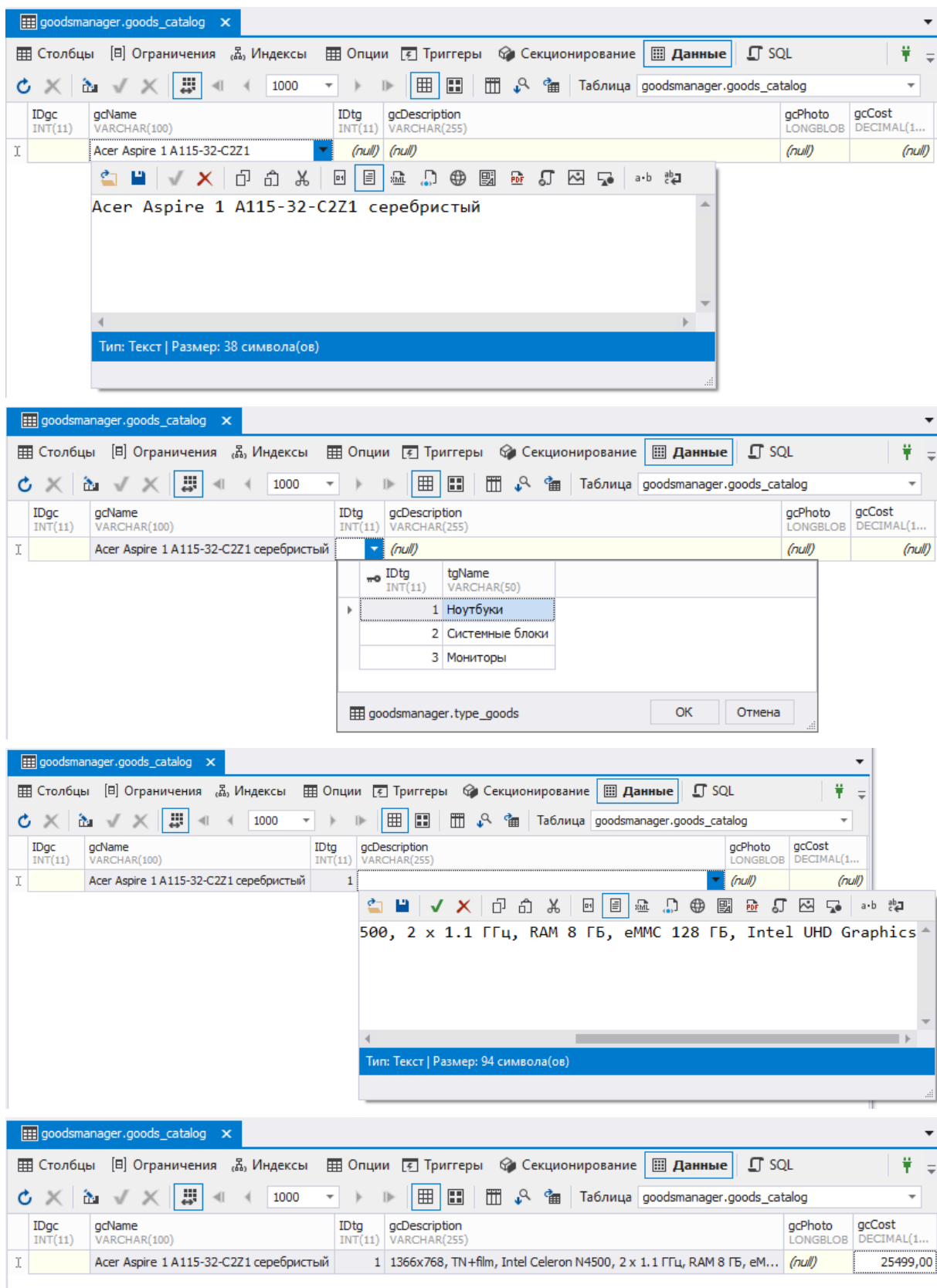


Рис. 1.28. Поэтапная демонстрация заполнения записи таблицы `goods_catalog`

Как можно заметить при выборе типа товара (заполнения поля `IDtg`) показываются данные из таблицы `type_goods` на основании внешнего

ключа. В таком виде заполнение таблиц становится значительно проще. Результат заполнения записей для **goods_catalog** представлен на рис 1.29

IDgc INT(11)	gcName VARCHAR(100)	IDtg INT(11)	gcDescription VARCHAR(255)	gcPhoto LONGBLOB	gcCost DECIMAL(11,2)
1	Acer Aspire 1 A115-32-C2Z1 серебристый	1	1366x768, TN+film, Intel Celeron N4500, 2 x 1.1 ГГц, RAM 8 Гб, е...	(null)	25499,00
2	Lenovo IdeaPad 3 15IGL05 серый	1	1920x1080, TN+film, Intel Celeron N4020, 2 x 1.1 ГГц, RAM 8 Гб, S...	(null)	27955,00
3	ASUS VivoBook M513UA-BQ080 черный	1	1920x1080, IPS, AMD Ryzen 7 5700U, 8 x 1.8 ГГц, RAM 8 Гб, SSD ...	(null)	49999,00
4	DEXP Mars E317	2	Intel Core i3 9100F, 4x3600 МГц, 8 Гб DDR4, GeForce GTX 1050 Ti,...	(null)	41999,00
5	ASUS ROG Strix GL10CS-RU077T	2	Intel Core i5 9400F, 6x2900 МГц, 8 Гб DDR4, GeForce GTX 1650, S...	(null)	64990,00
6	Samsung F27T450FQI	3	1920x1080@75 Гц, IPS, 5 мс, 1000:1, 250 Кд/м², 178°/178°, HDMI...	(null)	15199,00

Рис. 1.29. Пример заполнения данными таблицы *goods_catalog*

Создадим две записи фирм-поставщиков в таблице **suppliers_catalog**. Поскольку появились сведения о поставщиках, то теперь мы можем заполнить таблицу **operations** (рис. 1.30). Добавим несколько записей операций продажи товара и хотя бы одно поступление.

IDoperation INT(11)	oDateTime DATETIME	oIsSale TINYINT(1)	IDsc INT(11)
1	06.05.2021 11:25:00	1	(null)
2	06.05.2021 14:27:00	1	(null)
3	07.05.2021 11:15:00	0	

IDsc INT(11)	scName VARCHAR(50)	scAddress VARCHAR(100)	scPhone VARCHAR(15)	scEmail VARCHAR(50)
1	ТехноМаркет	Москва, пр. Ленинский 14	4953222517	info@tehnomark.ru
2	КрасТрейд	Красноярск, пр. Мира 31	3912223517	sales@krastrade.com

Рис. 1.30. Пример заполнения данными таблицы *operations*

Используя данные связанных таблиц, теперь можем приступить к заполнению таблицы **op_goods_list** соответствующими сведениями.

Для того чтобы можно было вести складской учет (хранить информацию о количестве товаров) нам необходимо в таблицу **storage** занести данные обо всех товарах, имеющих в каталоге **goods_catalog**.

Каждый раз при добавлении в каталог товаров (**goods_catalog**) нового товара придется добавлять информацию о нем в таблицу **storage**. Выполнять такие действия вручную не совсем целесообразно. Для этого можно воспользоваться механизмом триггеров.

1.6 РАЗРАБОТКА ПРОСТЫХ ТРИГГЕРОВ В MySQL

Триггер – это хранимая откомпилированная SQL-процедура, которая не вызывается непосредственно, а выполняется при наступлении определенного события внутри базы данных: вставки, удаления, обновления записей. Достоинством триггеров является то, что они могут сократить сетевой трафик. Сложные повторяющиеся задачи обрабатываются на сервере баз данных без необходимости отсылки промежуточных результатов приложению.

Триггеры применяются для обеспечения целостности данных и реализации сложной бизнес-логики. Они запускаются сервером автоматически при попытке изменения данных в таблице, с которой они связаны. Все производимые ими модификации данных рассматриваются как выполняемые в транзакции, в которой произошло действие, вызвавшее срабатывание триггера. И в случае обнаружения ошибки или нарушения целостности данных может произойти откат этой транзакции.

Триггеры выполняются автоматически при событиях **UPDATE**, **DELETE** и **INSERT** с возможностью выбрать момент срабатывания: до или после возникновения события. В триггерах к столбцам таблицы можно обращаться, используя псевдонимы **OLD** и **NEW**.

OLD.col_name ссылается на существующую строку столбца с именем **col_name** прежде, чем она будет модифицирована или удалена. **NEW.col_name** ссылается на новую строку столбца с именем **col_name**, которая будет вставлена, или на уже существующую строку после того, как она будет модифицирована.

Рассмотрим процесс создания триггера на примере триггеров для таблицы **goods_catalog**, которые будут отвечать за добавление/удаление информации о товаре в таблице **storage**.

Создание триггера можно инициировать двумя путями. Первый способ это через контекстное меню каталога **Триггеры** в проводнике, а второй – через контекстное меню, при редактировании таблицы находясь на вкладке **Триггеры** (рис. 1.31).

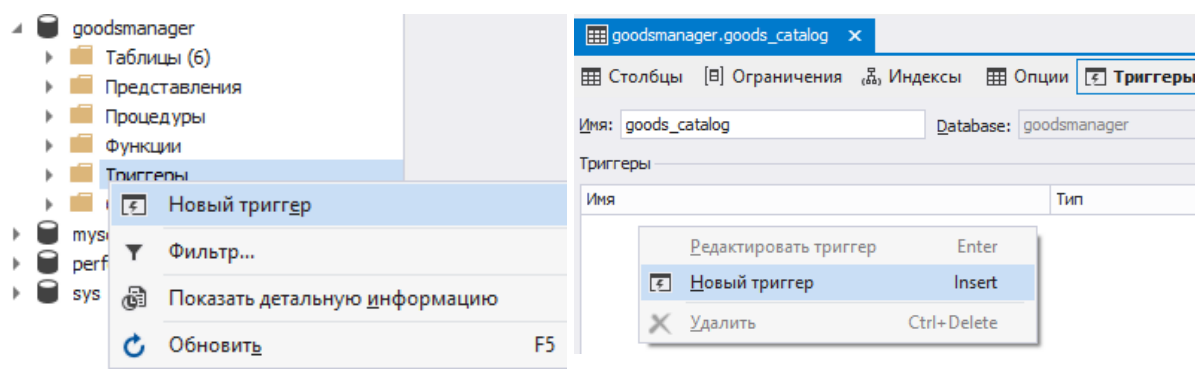


Рис. 1.31. Способы создания триггеров.

Создадим триггер, который будет отвечать за постановку на учет товара на склад. Данный триггер должен срабатывать после вставки записи в таблицу **goods_catalog** (рис. 1.32):

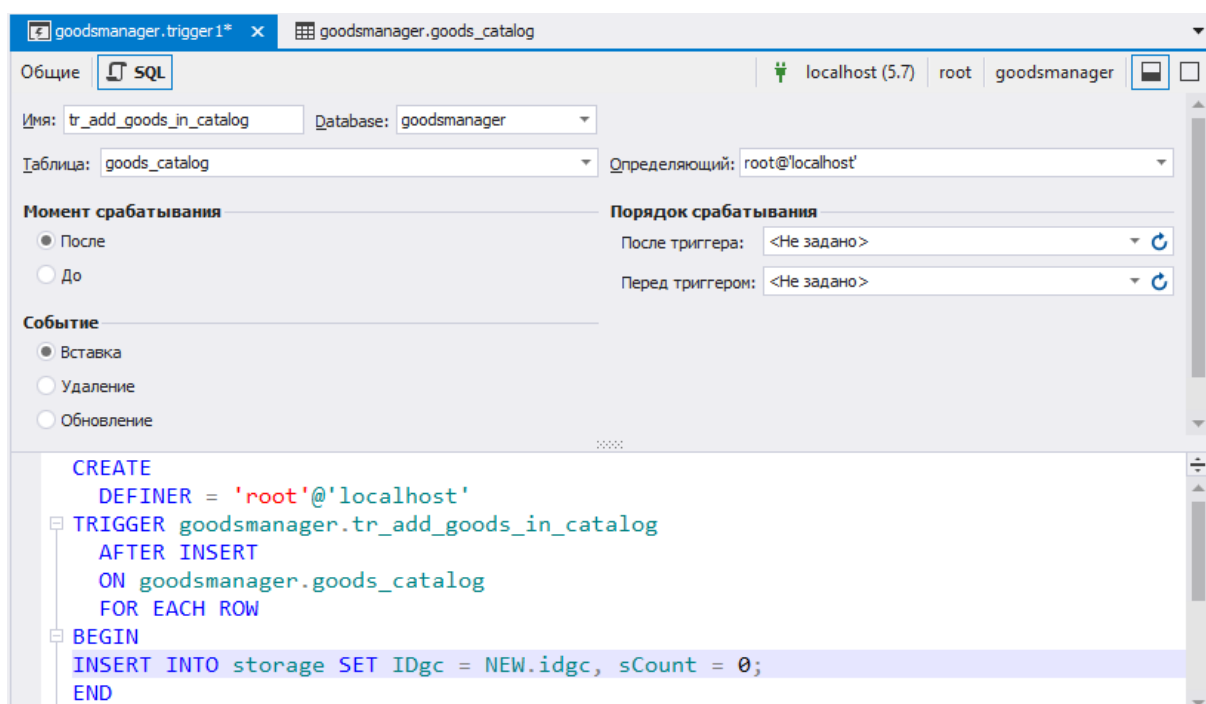


Рис. 1.32. Пример настройки триггера на добавление данных

Весь основной код, который пишет разработчик, располагается между строками **BEGIN** и **END**, остальная часть генерируется средой администрирования в зависимости от заданных настроек в интерфейсе. В нашем случае сформирована *SQL*-команда:

```
INSERT INTO storage SET IDgc = NEW.idgc, sCount = 0;
```

Она отвечает за добавление новой записи в таблицу **storage**, при этом полю **IDgc** устанавливается значение идентификатора нового товара, а полю **sCount** значение **0** (количество нового товара на складе равно нулю).

При использовании внешних ключей нет возможности удалить запись при наличии зависимых записей. В связи с этим на удаление товара из каталога также создадим триггер. Данный триггер будет срабатывать до удаления товара из каталога (рис. 1.33). Он будет удалять из перечня товара на складе только те товары, количество которых равно **0**. Для этого будет использован *SQL*-код:

```
DELETE FROM storage WHERE (IDgc=OLD.IDgc) AND (sCount=0);
```

Согласно *SQL*-коду будет выполняться удаление записи из таблицы **storage** при условии, что значение поля **IDgc** равно значению идентификатора удаляемого товара, а также **sCount** (количество товара на складе)

равно 0. При этом если товар участвовал в каких-либо операциях информацией о товаре нельзя удалить.

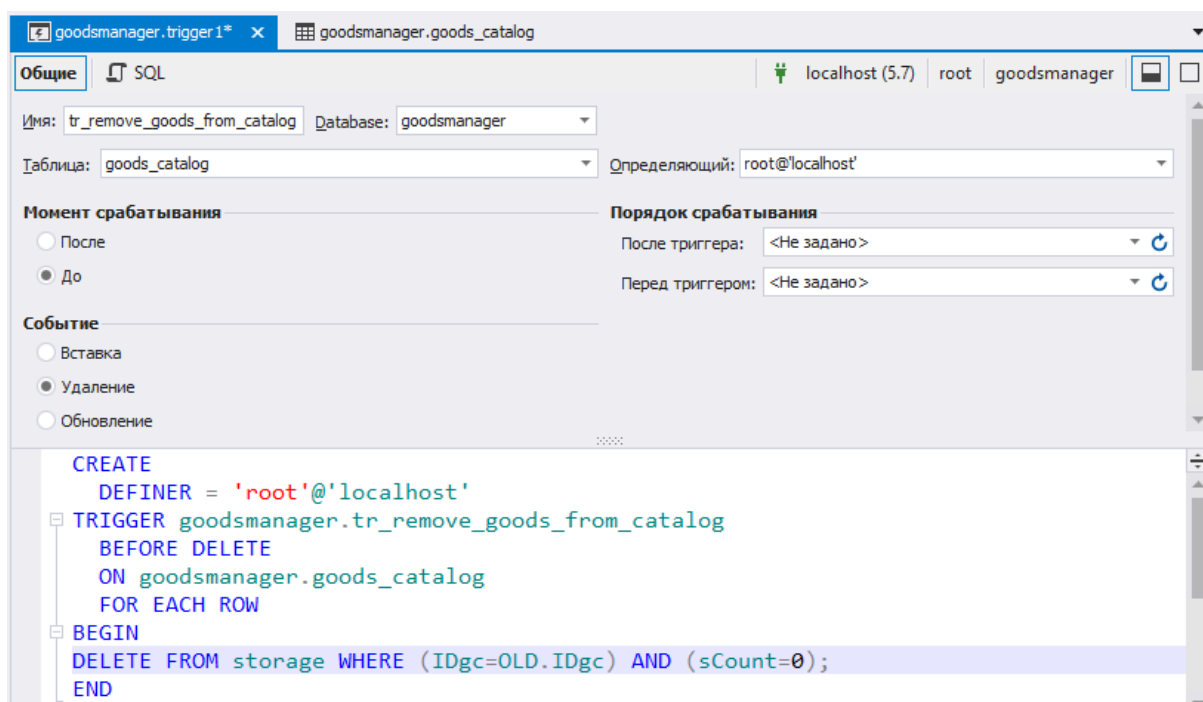


Рис. 1.33. Пример настройки триггера на удаление данных

Обращаем ваше внимание. Если вы добавляли товары в **goods_catalog** и **не добавляли** их количество в **storage**, то для корректной работы ключевых элементов бизнес логики вам потребуется внести сведения в таблицу **storage**.

Для вновь добавляемых товаров информация, при помощи триггеров, будет автоматически заноситься в таблицу **storage**. Добавим еще один товар и убедимся в этом.

1.7 СОЗДАНИЕ ПРЕДСТАВЛЕНИЙ

Представлением (*view*) называется *SQL*-запрос на выборку, который сохраняется как отдельный объект базы данных. Представление позволяет пользователю увидеть результаты сохраненного запроса, а язык *SQL* обеспечивает доступ к этим результатам таким образом, как если бы они были реальной таблицей базы данных.

Представления используются по следующим причинам:

- позволяют сделать так, чтобы разные пользователи базы данных могли видеть данные по-разному;
- упрощают синтаксис последующих запросов;
- они упрощают доступ к базе данных, показывая каждому пользователю структуру хранимых данных в наиболее подходящем для него виде.

В качестве пример создадим два представления, которые понадобятся в будущих лабораторных работах. Сгенерируем их при помощи визуального конструктора, который вызывается при помощи кнопки **Новый запрос** (рис. 1.34).

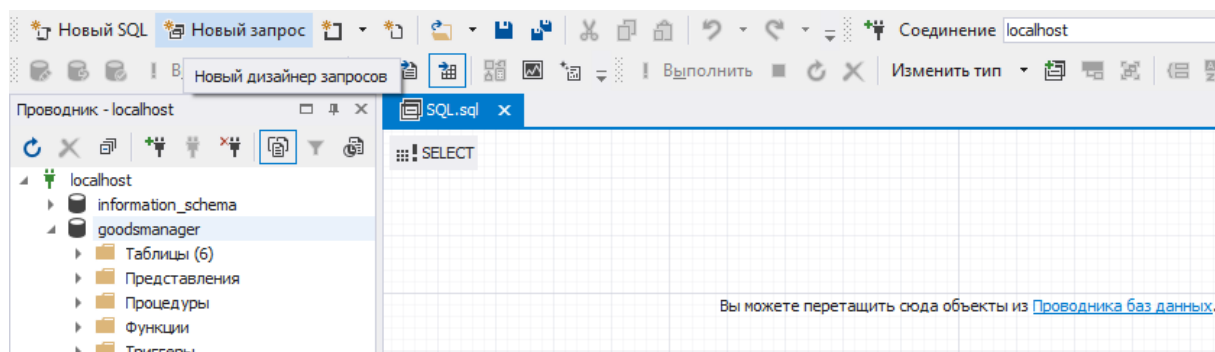


Рис. 1.34. Создание нового SQL-запроса, вызов конструктора

Сначала создадим запрос, отображающий каталог товара, где вместо поля **IDtg** будет отображаться название типа (категории). Для этого перенесите на пустое поле таблицы: **goods_catalog**, **type_goods** и выберите необходимые поля, как показано на рис. 1.35.

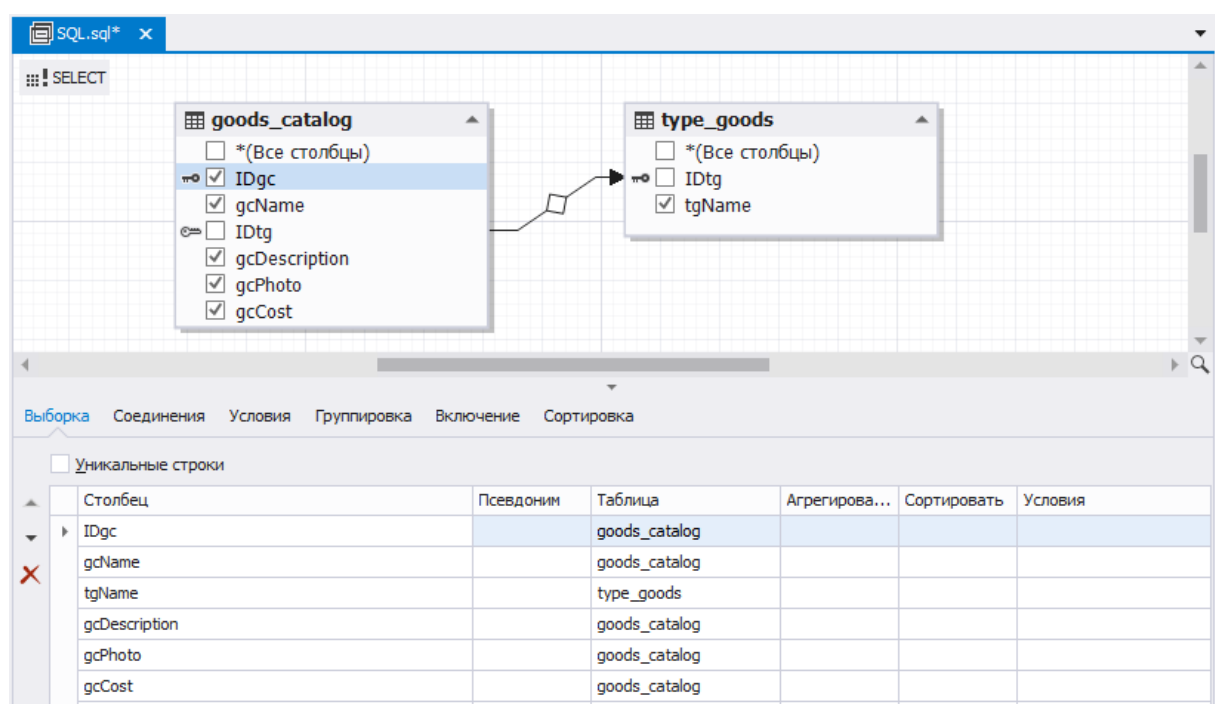


Рис. 1.35. Выбор необходимых полей таблицы

Для того чтобы просмотреть SQL-код сформированный конструктором, необходимо нажать на кнопку **Текст**. При создании представления будет использован этот SQL-запрос (рис. 1.36), поэтому скопируйте его.

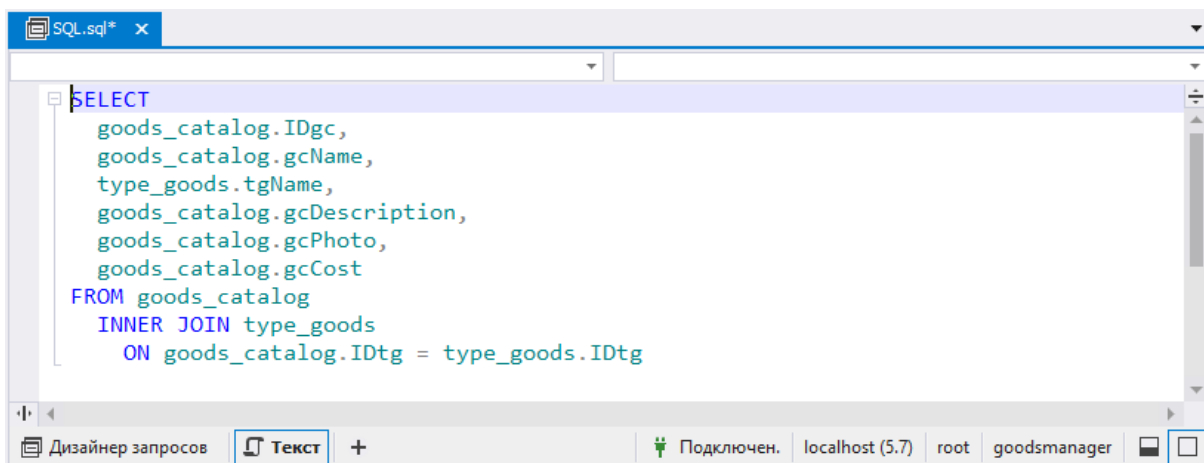


Рис. 1.36. Просмотр текста SQL-запроса

Для чтобы создать представления необходимо использовать контекстное меню, вызванное на папке **Представления**. Создадим новое представление, и дадим ему имя **view_goods_catalog**, после чего вставим в область **Текст запроса** ранее сгенерированный SQL-запрос (рис. 1.37). Для сохранения представления нажмем кнопку **Применить изменения**.

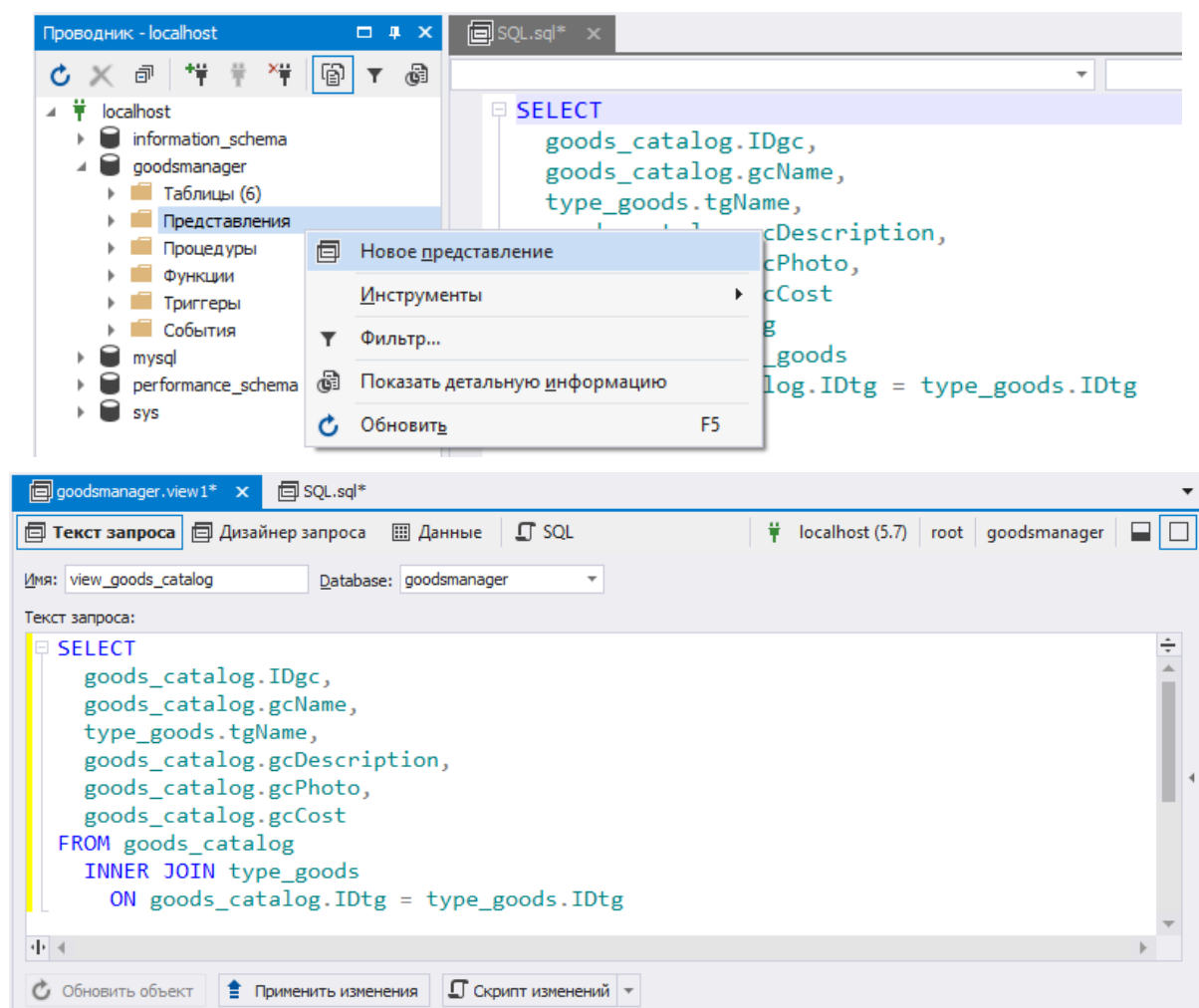


Рис. 1.37. Этапы создания представления

Теперь если нажать на кнопку **Данные** в представлении, можно увидеть результат выполнения SQL-запроса (рис. 1.38).

IDgc INT(11)	gcName VARCHAR(100)	tgName VARCHAR(50)	gcDescription VARCHAR(255)	gcPhoto LONGBLOB	gcCost DECIMAL(10, 2)
1	Acer Aspire 1 A115-32-C2Z1 серебристый	Ноутбуки	1366x768, TN+film, Intel Celeron N4500, 2 x 1.1...	(null)	25499,00
2	Lenovo IdeaPad 3 15IGL05 серый	Ноутбуки	1920x1080, TN+film, Intel Celeron N4020, 2 x 1.1...	(null)	27955,00
3	ASUS VivoBook M513UA-BQ080 черный	Ноутбуки	1920x1080, IPS, AMD Ryzen 7 5700U, 8 x 1.8 ГГц...	(null)	49999,00
4	DEXP Mars E317	Системные блоки	Intel Core i3 9100F, 4x3600 МГц, 8 ГБ DDR4, GeF...	(null)	41999,00
5	ASUS ROG Strix GL10CS-RU077T	Системные блоки	Intel Core i5 9400F, 6x2900 МГц, 8 ГБ DDR4, GeF...	(null)	64990,00
6	Samsung F27T450FQI	Мониторы	1920x1080@75 Гц, IPS, 5 мс, 1000:1, 250 Кд/м²,...	(null)	15199,00
7	HP V28 4K	Мониторы	3840x2160@60 Гц, TN, 1 мс, 1000:1, 300 Кд/м², ...	(null)	19399,00

Рис. 1.38. Вывод данных представления

Аналогичным образом создадим еще одно представление с именем **view_goods_operations** на основе SQL-запроса, показанного на рис. 1.39. При использовании визуального конструктора для формирования запроса у связи таблиц **operations** и **suppliers_catalog** нужно в контекстном меню выбрать опцию **Выбрать все строки из "operations"** (тем самым в SQL-запрос добавляется оператор *LEFT OUTER JOIN*). В таком случае если в поле **IDsc** будут присутствовать данные, то в формируемом результате будут отражены значения **scName**, иначе выводимое поле будет содержать значение **null**.

IDoperation INT(11)	oDateTime DATETIME	oIsSale TINYINT(1)	scName VARCHAR(50)
1	06.05.2021 11:25:00	1	(null)
2	06.05.2021 14:27:00	1	(null)
3	07.05.2021 11:15:00	0	ТехноМаркет

Рис. 1.39. Текст запроса и вывод данных представления *view_goods_operations*

Для отображения сведений о состоянии складских запасов в понятном для пользователя формате создадим представление *view_storage* текст запроса которого показан на рис. 1.40.

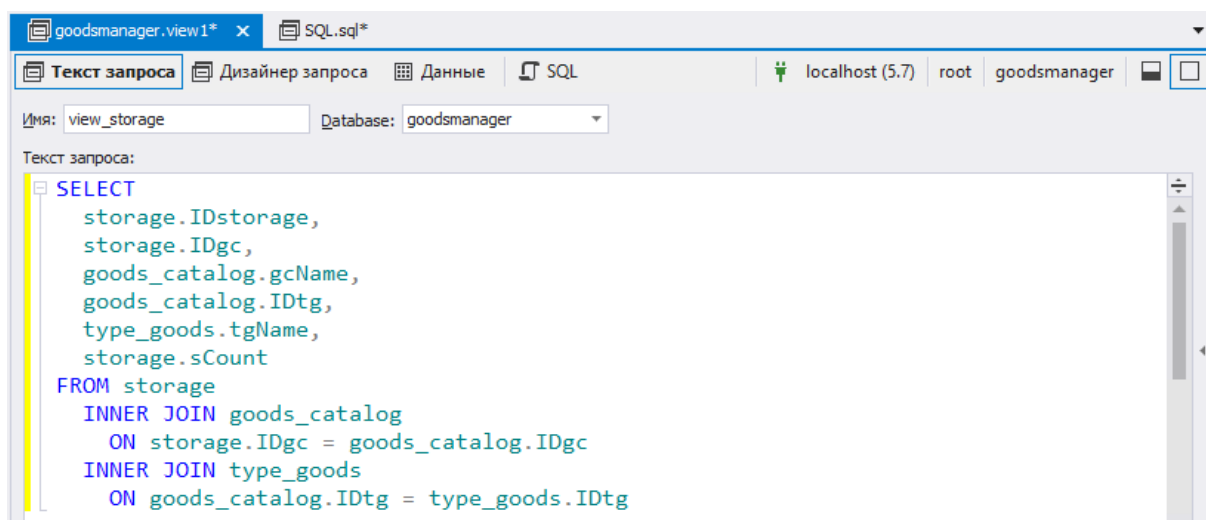


Рис. 1.40. Текст запроса представления *view_storage*

На данном этапе создание базы данных магазина компьютерной техники завершено. Теперь необходимо создать резервную копию базы данных (рис. 1.41), которая позволит не только восстановить данные в случае сбоя, но и перенести их в другую реляционную СУБД помимо *MySQL*.

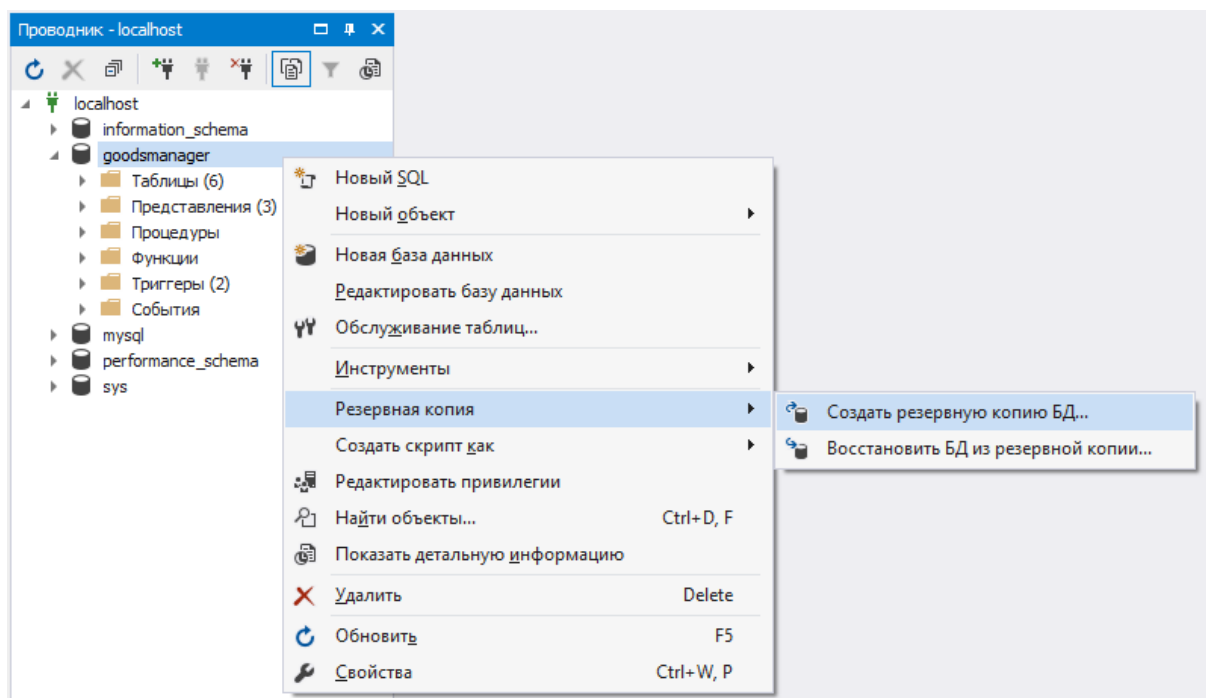


Рис. 1.41. Контекстное меню создание резервной копии БД

В появившемся окне мастера создания резервной копии БД достаточно указать путь сохранения резервной копии и нажать кнопку **Выполнить**. При желании также можно настроить параметры создания резервной копии, т.е. что будет включено в файл резервной копии.

Сама резервная копия представляет собой обычный текстовый файл содержащий *SQL*-код и имеющий расширение **.sql**. Открыв файл с резервной копией, вы найдете в нем, как сохраняются данные из БД.

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что из себя представляет GPL лицензия?
2. Какая информация хранится в БД *mysql*?
3. Дайте характеристику *UTF-8*, чем она отличается от *utf8mb4*?
4. Чем *InnoDB* отличается от *MyISAM*?
5. Что такое *ACID*?
6. О чем говорит количество отображаемых значений для типа данных?
7. Охарактеризуйте типы данных с фиксированной точкой.
8. Что такое *UTC* и *timestamp* (что будет с этим типом данных '2038-01-09 03:14:07')?
9. Опишите тип данных *BLOB*, для чего он нужен?
10. Для чего создаются уникальные индексы. Для чего предназначены индексы?
11. Что из себя представляют внешние ключи?
12. Какие правила обновления/удаления связанных данных вы знаете?
13. Почему не стоит для правила обновления выставлять *CASCADE*?
14. Дайте характеристику триггерам, каково их назначение?
15. Что из себя представляют псевдонимы *OLD* и *NEW*?
16. В какой момент срабатывают триггеры?
17. Охарактеризуйте назначение механизма представлений.
18. Какие виды операции *JOIN* существуют в *SQL*?
19. Что из себя представляет резервная копия реляционной базы данных?